

How the Work Changed

A work report on the FL-ADTCN paper extension

*Blockchain-Integrated Federated Fraud Detection —
what the accepted thesis got right, what it got wrong,
and what six days of controlled re-testing changed about both.*

Shadman Sakib (21301566)

with Md. Abrar-ud-doula Amiya • Mahima Islam • Nafi Alif Mahin • Muntasir Mahin Siam

Supervisor: Md. Golam Rabiul Alam

Reporting period: 30 August – 5 September 2026 • Compiled 5 September 2026

The short version — in plain language

What this project is. Three banks want to catch fraud. None of them is allowed to show its customers' transactions to the other two. So instead of sharing data, they each train their own detector and share only the *trained numbers* (the model weights). A blockchain keeps the record of who helped and who gets paid. The whole thing is called **FL-ADTCN**.

Where we stood on 30 August. The thesis had already been accepted. It had one genuinely new idea, an honest list of things that did not work, and one big weakness underneath everything: **every single claim had been tested on one dataset only** — and that dataset does not tell you which customer made which payment.

What we did in these six days, and why:

- **Added a second dataset (BankSim), and kept the first one.** *Why:* a claim that only holds on one file might be a fact about that file. Testing on two lets us tell the difference.
- **Ran a “control” experiment.** A control changes *one* thing at a time so you can tell which thing actually caused an effect. Ours changed only the dataset and kept everything else fixed. *Why:* our most exciting new finding had changed two things at once, so we could not know which one was responsible. *Result:* *the control proved us wrong and we withdrew the claim.*
- **Audited the search algorithm the framework is named after (DB-BOA).** *Why:* five completely different search algorithms had all returned exactly the same score, which is not something that happens by chance. *Result:* the search had been marking its own homework — it was testing itself on rows it had already trained on. We fixed it, re-ran it properly for 16.3 hours, and the original bad news about it survived.
- **Stopped quoting one of our own numbers** — the “privacy improvement factor”. *Why:* we proved the number *grows the harder you look for it*, so it describes our search effort, not the system. A number like that cannot safely be printed in a paper.
- **Put all of it into version control.** *Why:* before 5 September roughly 20 hours of results existed only as untracked files on one machine.

What it added up to. Three of our own headline claims died in six days, and all three were killed by tests we ran on ourselves — not by a reviewer. That is the point of the period. The work is smaller than it looked on 30 August, and far harder to argue with.

How to read this report. It runs in order, as a story. If you only have five minutes, read the box above and then Section 8, the decision log. If you are about to write something and need to know which numbers are safe, go straight to Section 11. If a score or a term is unfamiliar, Section 2 explains the scores and Appendix A is a full glossary.

Contents

1	What this document is	4
2	How to read the numbers in this report	4
2.1	Why plain accuracy is almost useless to us	5
3	The system, in one page	5
4	Where we stood on 30 August — what was good	7
5	Where we stood on 30 August — what was bad	8
6	How we decided to work	10
6.1	The standing rules	10
6.2	Pre-registration, and what it bought	10
7	The work, in order	11
7.1	First, close the three loose ends a reviewer would find (31 August)	12
7.2	The dataset question — and the decision to keep ULB	12
7.3	All five datasets, compared	13
7.4	What the second dataset said about our own detector (31 August)	15
7.5	Taking the system claims off one dataset (1 September)	17
7.6	The control that killed our best new claim (1–4 September)	17
7.7	Propagating a withdrawal is its own job (4 September)	19
7.8	The privacy budget: we measured it, and then retired it	20
7.9	The search the framework is named after (4–5 September)	22
8	The decision log	27
9	Where the six claims stand now	28
10	The comparisons, side by side	29
10.1	What changed when the dataset changed	31
10.2	The score index — every metric, every condition	32
10.3	The two negative results, tested	33
10.4	The system, measured	34
11	What may be quoted, and what may not	34
12	What is left, and the plan to 19 September	35
12.1	Open questions we are carrying honestly	36
13	Closing assessment	36
A	Glossary	38
B	Where everything lives	39

1. What this document is

This is a work report, not a paper. It is written for two readers: a teammate who has to pick the work up, and a supervisor who needs to know what changed and whether to trust it. Read it in order — it is written as a story. Where we stood, what was wrong with where we stood, what we decided to do about it, what happened when we did it, and what we are left with.

You do not need to already know the field to read it. Every technical term is explained the first time it appears, and there is a full glossary in Appendix A. Where a section depends on understanding a score, the score is explained right there rather than assumed.

Three things this document does on purpose:

- **It names the decisions, and who made them.** Several times in these six days the work could have gone two different ways. Each of those forks is printed in a blue box, with four things: the question we faced, the answer we chose, the reason we chose it, and — the important part — what that answer actually produced, *including the times it produced something worse than the road not taken*.
- **It reports failures as results.** Before each experiment we wrote down what we expected to happen. Nine of those written-down predictions turned out to be wrong. Those nine are the most useful material in the whole period, so they are printed next to their original wording instead of being quietly replaced with what actually happened.
- **It marks the numbers that may no longer be used.** Several figures that appear in older drafts and in the supervisor brief are now either invalid or depend on an arbitrary choice. Section 11 lists them, with what to say instead.

Status at a glance, 5 September 2026

Phase	Thesis accepted 13 June 2026. Paper extension active. Deadline 19 September 2026 — 14 days away.
Datasets assessed	Five , compared side by side in Section 7.3: ULB, BankSim, Fraud Detection Handbook, IBM AMLSim, PaySim.
Datasets measured	Two . ULB credit-card (284,807 transactions, 0.17% fraud, <i>no customer IDs</i>) and BankSim (594,643 transactions, 1.211% fraud, 4,112 customers). The Handbook is loaded and verified but unrun — go/no-go 12 September; AMLSim and PaySim were assessed and not run.
Conditions measured	Three : ULB/stratified, BankSim/stratified, BankSim/entity-disjoint. The third is a <i>control</i> — it exists only to pull apart two effects that used to move together.
Promises tested on more than one dataset	Five of six . The sixth group (RL, consensus, blockchain) does not depend on the dataset at all, by design.
Computer time spent	Roughly 45 hours of experiments, of which one deliberately expensive run took 16.3 hours on its own.
Claims we withdrew or re-tired	Three , and all three were killed by our own tests: the explanation for Krum’s accuracy cost, “the fast Shapley estimate fails from $n \approx 6$ ”, and the privacy improvement factor as a headline number.
Code state	Everything committed on branch obj13-surrogate-repair . Before 5 September the entire extension existed only as untracked files on one machine.

2. How to read the numbers in this report

This report quotes a handful of scores over and over. This section says what each one means, why we chose it, and what a good or bad value looks like. If a number later in the document does not make sense, the explanation is here.

2.1 Why plain accuracy is almost useless to us

Fraud is rare. In our first dataset only **0.17%** of transactions are fraud. So a detector that simply says “not fraud” to *every single transaction* — catching nothing at all — scores **99.83% accuracy**. That number looks excellent and the detector is worthless.

This is not a hypothetical worry. It is exactly what bit us: in Section 9 a privacy-protected model is destroyed so badly that it becomes useless, and plain accuracy reports the damage as -0.11 percentage points. **Accuracy hides failure on rare-event data.** That is why we lead with MCC instead.

Score	Range, and what good looks like	What it actually measures, in plain terms
MCC (Matthews correlation coefficient)	-1 to $+1$. 0 = no better than guessing. Our best ≈ 0.78 .	Our headline score. It asks: does the detector’s “fraud / not fraud” answer actually correlate with the truth? It only goes up if the model gets <i>both</i> classes right, so the “say no to everything” trick scores 0 , not 99.83%. This is why we use it.
Balanced accuracy	0 – 100% . 50% = guessing.	Accuracy worked out separately for fraud and for non-fraud, then averaged — so the rare class counts as much as the common one. Used in the attack experiments. <i>Warning: it can be fooled too — see the box in Section 7.6.</i>
Accuracy	0 – 100%	The plain percentage of transactions labelled correctly. We report it <i>only</i> to show how misleading it is on data this imbalanced. Never used as a headline.
Precision / Recall	0 – 1 each	<i>Recall</i> : of all the real frauds, how many did we catch? <i>Precision</i> : of everything we flagged as fraud, how much really was? A model can max out one by wrecking the other, which is why neither is a headline on its own.
pp (percentage points)	—	The gap between two percentages. Going from 90% to 85% is a fall of 5 pp . We use “pp” so a difference is never confused with a percentage <i>of</i> a percentage.
ρ (Spearman rank correlation)	-1 to $+1$	Are the three banks <i>ranked</i> in the right order by how much they contributed? $+1$ = perfect order, 0 = random order, -1 = exactly backwards. This matters because the banks get paid according to that ranking.
ε (epsilon, the privacy budget)	small = private	The dial on the privacy protection. Smaller ε means more privacy and more added noise , so more damage to the model. Larger ε means less privacy and a more useful model. The whole privacy question is where to set this dial.
Latency / TPS	ms / transactions per sec	Blockchain speed. <i>Latency</i> is how long one record takes to be written and confirmed (ours: 2115.9 ms). <i>TPS</i> is how many we can write per second (ours: 40.3 at best).

One habit that runs through the whole report: say which score you mean

Twice in these six days, two perfectly reasonable scores ranked the same experiment in *opposite* directions. If you quote the flattering one without saying which one you picked, that is not reporting — it is selecting. So every claim in this document names its score. It is the difference between a result and a choice.

3. The system, in one page

Before anything else it has to be clear what the “thing” is that we are judging — because one of the mistakes we made in this period was judging the entire thesis by one small piece of it.

The thing is FL-ADTCN, a six-part system — not a classifier. Here is the whole idea in everyday terms. Three banks (call them A, B and C) each want a better fraud detector. Each bank has its own transactions and is not allowed to hand them to the other two. So instead of pooling the data, each bank trains a detector on its own transactions and shares only the *trained numbers* inside that detector — the model weights. Nobody’s customer data ever leaves the building.

That much is standard federated learning. Around that weight-sharing step our system adds five more mechanisms, and each one answers a different worry:

- **Differential privacy** — “*could someone reconstruct my customers from the numbers I shared?*” Deliberate random noise is added to the weights before they are shared.
- **Krum** — “*what if one of the banks is lying?*” A rule that inspects all the submitted weight sets and picks the single most typical one, so a sabotaged submission is never used.
- **Shapley values** — “*who actually helped, and how much should they be paid?*” A fairness calculation that splits the credit between the banks.
- **DB-BOA** — “*what settings should the detector use?*” An automated search over the detector’s configuration, so the settings are not hand-guessed.
- **A Hyperledger Fabric blockchain** — “*who keeps the books, and can we trust them?*” A shared tamper-evident ledger records the payments, and picks which bank leads each round.

FedAvg — the baseline everything else is measured against

FedAvg (Federated Averaging) is the standard, original way to combine what the banks send back, and it is the yardstick we compare against all through this report. It is exactly as simple as it sounds: **take every bank’s weights and average them, number by number.** If Bank A submits 0.4, Bank B submits 0.6 and Bank C submits 0.5 for the same weight, the global model gets 0.5. Do that for every weight in the model, send the averaged model back to all three banks, and repeat next round. (In our code it is a plain, unweighted mean — `byzantine_robustness_sweep.py:102.`)

Its strength is that it is simple and it works. Its weakness is that an average has no defence against a liar. One bank submitting deliberately enormous numbers drags the average wherever it likes, and the other two cannot outvote it, because averaging is not a vote — everyone gets included by construction. That single weakness is the reason Krum exists.

So whenever this report writes “**Krum – FedAvg**”, it is asking one question: *what do we gain, or lose, by picking the most typical bank instead of averaging all of them?* A positive number means Krum’s protection also made the model better. A negative number means the protection cost us accuracy. Both happen, and Section 7.6 is about finding out why.

The thesis title makes six promises, and **a different part of the system is responsible for each one.** This table is the map. It matters because if you review only one component, you have reviewed only one promise — which is precisely the mistake described in the red box below.

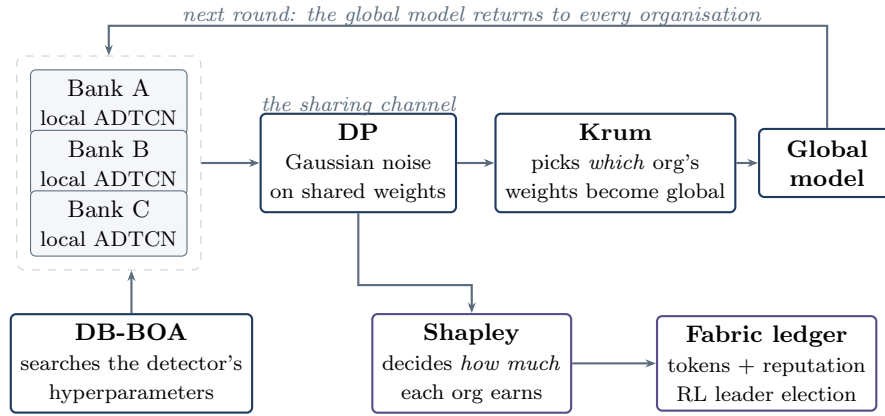


Figure 1: FL-ADTCN. Raw transactions never leave an organisation — only weights are shared, and only after noising. Krum and Shapley are *independent by design*: Krum is security (whose weights are used), Shapley is fairness (who gets paid). A Krum-rejected organisation can still earn tokens. They are not two stages of one pipeline, and describing them that way is a recurring error.

Component	Its job, in plain terms	Which promise it backs
Federated ADTCN	the detector each bank runs, and the channel they share weights over	the foundation — backs no promise by itself
Differential privacy	adds noise so shared weights cannot be traced back to customers	Secure (the privacy half)
Krum	throws out a bank whose weights look sabotaged	Secure (the attack half)
Shapley values	works out who contributed what, so the rewards are fair	Incentivized, Scalable
DB-BOA	searches automatically for the detector's settings	automation — backs no promise directly
RL leader + Fabric	learns which bank should lead each round, and records it on-chain	RL, Blockchain-Integrated, Consensus

A mistake we made ourselves — written down because it cost a day

On 1 September we ran an internal review of our own direction, and it judged the *entire* thesis by a single thing: where our detector, ADTCN, ranked against six other detectors. **Every number in that review was correct, and its conclusion was completely wrong.**

Here is why. Look at the table above: ADTCN's ranking backs *none* of the six promises. It is the foundation the other parts sit on, not a promise in itself. So a review built on that one ranking silently threw away four of the six promises, and then recommended rewriting the paper around the two that happened to be left.

The correction — **judge the whole system, not one component** — became a standing rule, and it turned the recommendation completely round: the dataset work was *under-done*, not *over-done*. Everything in Section 7 follows from that reversal.

4. Where we stood on 30 August — what was good

Before the thesis was submitted it went through one painful audit of its own work, and that audit is the reason the rest of it can be believed. Six things were already solid on 30 August.

G1. The central new idea is real, specific, and nobody else had described it. Here it is in plain terms. The banks add noise to their shared weights to protect privacy. Separately, the

system works out how much each bank contributed, and pays them accordingly. **What nobody had noticed is that the first of these breaks the second.** The privacy noise scrambles the contribution calculation so badly that the ranking of who helped most becomes meaningless — so honest banks get paid the wrong amounts. Our fix is to move the noise: instead of noising the *weights*, noise the *published contribution score*. That protects privacy just as well and leaves the ranking intact, at a fraction of the privacy cost. Two mechanisms that are normally studied in separate papers turn out to interfere with each other, and that interference is the thesis’s actual contribution.

- G2. The blockchain is really running, not simulated.** There is working chaincode on a real Hyperledger Fabric test network, and we measured it rather than estimating it: **2115.9 ms** average time to commit one record (over 50 transactions), and a best throughput of **40.3 transactions per second** at 10 concurrent clients. Push past that and it falls apart quickly — half the transactions fail at 20 concurrent clients, 70% at 40.
- G3. The attack defence was built where its maths actually applies.** Krum’s guarantee only holds when there are enough honest participants relative to attackers, so we ran it in that regime and swept four different attack types. **The attacker was caught 8 times out of 8.**
- G4. The work had already found and deleted its own invented numbers.** An earlier audit removed nine claims that no script had ever produced — a 97.38% accuracy, an MCC of 0.966, 85 TPS, 180 ms, a comparison table of eight models, some statistical tests, and two more tables. They were written but never computed. That clean-out is what makes the numbers that survived worth reading.
- G5. Results that came out badly were kept as results.** The automated search does not beat a setting we picked by hand. The fast approximation of the fairness calculation is not reliable at the sizes we can check. The reward system does not stop a lone attacker or a free-rider. Krum has no guarantee with only three banks. **None of these was hidden, softened or spun.**
- G6. Every headline number could be traced back to a script and a data file.** There was no figure in the report that could not be regenerated from scratch.

5. Where we stood on 30 August — what was bad

None of the weaknesses were sloppiness. They all had the same shape: **the evidence underneath a claim was narrower than the claim standing on it** — and in several cases you could not see how narrow it was from inside the project. That is the dangerous kind: it survives your own review because everything looks consistent, and it fails somebody else’s review because they ask where the second example is.

The weakness	Why it was a problem — what a reviewer would do with it
B1 All six promises rested on one dataset — ULB — and ULB does not say which customer made which payment.	Anything we reported could be waved away as a quirk of one anonymous file where only 0.17% of rows are fraud. And it was worse than that: our detector reads “sequences” of 10 consecutive transactions, but with no customer column those 10 rows belong to <i>ten different unrelated people</i> . We were testing a model designed to learn a person’s habits on input that contained nobody’s habits.
B2 Four of the six experiment scripts had the ULB file hard-wired into them , and none of the result files recorded which dataset they came from.	So testing on a second dataset was not merely something we had not done — it was not <i>possible</i> without writing code first. And results that do not record where they came from cannot be compared with anything later, even after you do that work.

The weakness	Why it was a problem — what a reviewer would do with it
B3 The detector’s headline score came from a single run , and two of our own files disagreed about it: one said MCC 0.677 , another said 0.313 .	A contradiction inside your own results, with no explanation, is the very first thing a reader finds — and once found it makes every other number on the page look unreliable too.
B4 The real, measured blockchain numbers were missing from the report. The invented ones had been deleted, but nothing had been put in their place.	Claiming “blockchain-integrated” while showing no measurement of the blockchain is an assertion, not a result.
B5 The automated search had never been checked. On ULB, all five different search algorithms we compared reported the same best score: exactly 5.0000 , which happens to be the highest score possible.	Five genuinely different algorithms agreeing to four decimal places tells you nothing about the algorithms. It tells you something is wrong with the <i>scoring function</i> they all share. And that meant the component the whole framework is named after had never been examined.
B6 The three “banks” were not really three banks — they were one bank’s transactions dealt into three equal piles.	Real banks differ from each other; three piles from one source do not. So there was nothing for the fairness calculation to find, and it duly reported all three as near-identical contributors ([0.167, 0.165, 0.165]). The reward layer had essentially nothing to reward.
B7 We quoted a privacy number as if we had measured it (“a 60× improvement”) when in fact our test had run out of range before finding the answer.	The true value was somewhere <i>beyond</i> where we stopped looking, so what we had was a floor, not a measurement — and we had printed it as a measurement. That is exactly the kind of overstatement the earlier clean-out existed to remove.
B8 Two of the six words in the title claim more than we delivered: “Scalable” (what scales is the <i>fairness calculation</i> , not the transaction throughput) and “Consensus Mechanisms” (we use Fabric’s stock Raft, we did not build one).	Reviewers read a title as a list of claims, and check them one by one.
B9 Numbers we had formally retracted kept reappearing in live drafts , because the only way to refresh a generated document was to re-run an experiment lasting hours.	So nobody refreshed them. A stale draft is cheaper to ignore than to fix, which means it stays broken indefinitely. This is a broken process, not a typo.

The “before” state in one sentence

The thesis had a genuine contribution and an honest list of things that did not work — sitting on top of **evidence from one dataset, a search nobody had ever checked, and one number we had presented as measured when we had not actually measured it**. All three are the kind of weakness that gets past your own review and does not get past anybody else’s.

6. How we decided to work

Before the experiments, the *method* — because changing how we worked was the single biggest change of these six days, and it is what produced most of the findings. The experiments themselves were fairly ordinary. The discipline around them was not.

6.1 The standing rules

A short list of rules governs every run. Here are the ones that actually did work in this period, in plain language:

- **Rule 1 — Be novel and be completely honest. If those two ever pull in opposite directions, honesty wins.** Every other rule is a consequence of this one.
- **Rule 2 — No number goes in a document unless a script produced it.** If the number does not exist yet, either run the experiment or delete the sentence. There is no third option.
- **Rule 3 — Bad results stay bad.** When something does not work, that *is* the finding. Do not go looking for a version of the experiment that makes it look better.
- **Rule 4 — Do not shop for results, and write your prediction down first.** Never pick a dataset, a random seed or a split because it makes us look good — and write down what you expect to happen *before* you run, so you cannot quietly adjust the expectation afterwards.
- **Rule 8 — A new dataset has to be able to prove something wrong, and covering all your claims beats piling up datasets.** One claim tested on three datasets while its four siblings sit on one is a lopsided table, and a reviewer will simply attack the four.
- **Rule 9 — Judge the whole system, not one component.** (Added after the review mistake described on the previous page.)
- **Rule 11 — Always say which score you are quoting, and why that one.** If a script computes two different numbers and the write-up quotes the nicer one without mentioning the choice, that is result-shopping wearing a disguise.
- **Rule 12 — An experiment that could destroy a finding you already have is worth more than one that could add a new finding.** (Added 4 September, immediately after a three-hour control run destroyed our best claim.)

6.2 Pre-registration, and what it bought

“Pre-registration” sounds formal but the idea is simple: **before starting a run, write down what you expect to happen and what it would mean if you turned out to be wrong — then never edit those words.** When the results arrive you write them underneath, next to the prediction, whatever they say.

The reason to do it is that human memory is generous. Without it, a surprising result quietly becomes “well, that is roughly what we thought” by the time it is written up. With it, being wrong is a recorded event you cannot talk your way out of.

Across five runs in this period we wrote down **26 predictions. Nine of them were wrong** — and those nine are the most valuable material in this report.

Run	Predictions	Wrong	The most useful failure
BankSim / entity-disjoint sweeps (1 Sep)	4	2	“MC top-1 fails from $n \approx 6$ ” — the premise itself was wrong; the failure is intermittent, not a threshold.
Confound control (1 Sep, scored 4 Sep)	6	2	Krum’s utility cost tracks the dataset , not the partition — and the pre-registration had named that exact alternative.
Privacy grid extension (4 Sep)	5	1	All three conditions de-censored, which we had predicted would not happen and had pre-named as the more valuable outcome.
Privacy repeat-count pass (4 Sep)	5	1	ULB’s output threshold did <i>not</i> rise; the falsifier fired verbatim as written.
Surrogate repair side-by-side (4–5 Sep)	6	3	Determinism does not make a noisy search reproducible — shared draws do. This was pre-registered the other way round.
Total	26	9	In six of the nine, the pre-registration had already written down what failure would mean.

Why this matters more than any single result in the report

Three of our own headline claims died in these six days. Because we had written down in advance what would count as being wrong, each of those deaths arrived as **a finding we could publish** rather than as an objection we had no answer to. The difference is entirely in the timing: the same fact, discovered by us first, is a contribution; discovered by a reviewer first, it is a retraction.

The control run that killed our best claim cost about three hours. Skipping it would have cost us the claim anyway — just later, and in public. **That trade is the real methodological result of the period.**

7. The work, in order

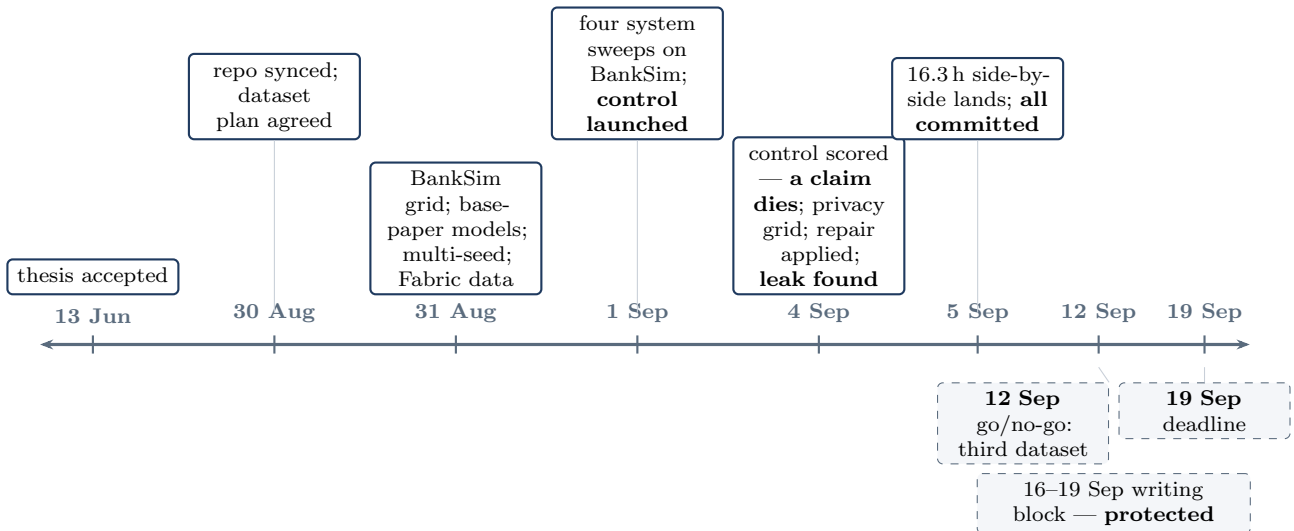


Figure 2: The period. Everything above the line has happened and is on disk; everything below it is planned. The 16–19 September writing block is protected: if the third dataset overruns, the dataset is cut, not the writing.

7.1 First, close the three loose ends a reviewer would find (31 August)

Before adding anything new, we closed the three problems from Section 5 that were cheapest to fix and most embarrassing to leave open.

First: the two contradictory scores (B3), and the answer was not what we had guessed. Recall the problem — one of our files said the detector scored MCC 0.677, another said 0.313, and nobody knew why. Thirteen full training runs settled it. Training is perfectly repeatable, *provided* three things stay fixed: the random seed, the version of the maths library, and **the number of CPU threads**. That third one was the culprit. Two of our scripts were using different thread counts, so they had never actually been doing the same arithmetic.

Why would thread count matter? Because adding up a million numbers in a different order gives a very slightly different answer in floating-point arithmetic, and over 30 training epochs those tiny differences compound. At the same seed, the tuned configuration scores **0.708, 0.746 and 0.800** on 4, 2 and 8 threads respectively. So our old guess — “large batches make training unstable” — had the right suspect and the wrong crime. **MCC 0.313 is withdrawn**: it does not reproduce under any condition we could construct.

The honest replacement is not a single number but a range over five runs: **tuned 0.753 ± 0.055 , hand-set default 0.706 ± 0.076 , a difference of $+0.047$ with $p = 0.34$** . In plain terms, $p = 0.34$ means a gap this size would turn up about a third of the time by pure chance, so we cannot claim the tuned version is better — and, awkwardly for us, the *default* is the more stable of the two. The underlying cause was that the maths library was never pinned to a version; it now is, and every detector number we report from here on has to state its library version and thread count or it means nothing.

Second: the real blockchain numbers went into the report (B4). The invented 85 TPS / 180 ms were replaced with the measured 2115.9 ms average latency and the 40.3 TPS curve. Measuring it also exposed something the simulated version had been hiding: **our reward scheme pays a 15-token bonus for committing a record in under 300 ms, and the real network’s floor is 2.1 seconds**. That bonus can never be earned by anyone. It was a design mismatch that only appeared once real hardware was involved.

Third: we deleted a duplicate definition. The scoring formula existed in two versions — an old unbounded one still sitting in the code, and the bounded one described in the methodology chapter. There is now exactly one, checked against the other over 2,000 random inputs and identical every time.

7.2 The dataset question — and the decision to keep ULB

This is the fork the whole rest of the work hangs on, so it is worth setting out carefully.

The problem. Our detector is built to spot suspicious *patterns over time* in a person’s spending. To do that it reads 10 consecutive transactions at once. But ULB has no customer column, so those 10 transactions belong to 10 unrelated strangers. **We had been testing a model designed to learn habits on data that contained no habits** — and it had duly lost to a much simpler model (0.770 against 0.459). We could not tell whether our architecture was genuinely bad or had simply never been given a fair test.

DECISION 1 — Add a second dataset, and *keep* the first one running too

Question raised: Do we **replace** ULB with a dataset that has customer identities, or **keep** ULB and run both side by side?

Decision taken: Keep ULB, add BankSim, and re-run *every* comparison on both, through the same code. BankSim was picked first out of the candidates because it is a single CSV file with 594,643 transactions, 7,200 frauds and 4,112 customers who each have about a hundred transactions — small enough to iterate on in minutes. (The full five-way comparison is

Section 7.3.)

Why: Three reasons, and the third is the one that mattered.

- (i) **ULB is what the accepted thesis stands on.** Drop it and every new result becomes incomparable with the submitted report, leaving that report stranded.
- (ii) **You can only show a result depends on the dataset if you measure both.** Swapping one dataset for another does not broaden the evidence — it just replaces one narrow story with a different narrow story.
- (iii) **Rule 4.** Keeping the dataset where our architecture looks good and dropping the one where it looks bad is result-shopping, however reasonable the excuse sounds at the time. This is exactly the situation the rule exists for, and it was tempting.

Outcome — and it genuinely cut both ways:

- **It paid immediately.** Entity-linked windows help *every* architecture by +0.0999 to +0.1717 MCC, so “temporal complexity does not pay” was a statement about entity-*less* windows, not about time. The differential-privacy collapse replicated on the second dataset — and its *direction* flipped, which neither dataset shows alone.
- **It also produced the two most uncomfortable findings of the period, and both are ULB-only.** Keeping ULB is the only reason we discovered that the deployed hyperparameter search was validating on memorised rows (Section 7.9), and the only reason we discovered that a ULB result from 30 August no longer reproduces (Section 7.8). Had we dropped ULB, both defects would still be in the work and neither would be known.
- **The honest cost:** two ULB result files are currently not quotable, and one reproduction failure is recorded as *unexplained*. Keeping ULB bought us the knowledge and the bill together.

A trap the second dataset nearly set — caught before it ran

BankSim’s raw file lists fraudulent transactions **next to each other** within each day — 3,635 adjacent fraud pairs in file order, against only 84 once you shuffle within the day. The reason is mundane: the simulator that generated the file wrote its frauds out together. It says nothing whatsoever about the real world.

But here is what it would have done to us. Our whole experiment compares customer-linked windows against ordinary sequential ones. If we had built the sequential windows straight from file order, the *comparison arm* would have found that a fraud is followed by another fraud 50% of the time — making it look strongly predictive, for a completely fake reason. **The comparison would have been poisoned and the entity-linkage result destroyed, and nothing would have looked wrong.**

Shuffling within each day fixes it, and the corrected numbers behave exactly as they should: sequential windows **0.0117** against a base rate of 0.0121 — no signal at all, which correctly matches what we see on ULB — and customer-linked windows **0.3615**.

The general lesson, which is now a mandatory check before any new dataset: **if a dataset’s time column is coarser than its row order, the row order is carrying hidden information that came from how the file was written, not from the world.**

7.3 All five datasets, compared

Decision 1 says which two datasets we run. This section says **which five we looked at**, what each one is, what each could and could not prove, and exactly how far each one got. Two are measured, one is wired up and ready but has not been run, and two were investigated and deliberately not run.

Read this before the table — what “compared” means here

Five datasets are compared on this page. Two of them have results. That distinction is the whole point of the table, so it is printed in its own column and never blurred. Under Rule 2, a dataset with no run behind it gets no numbers of its own — it gets its measured properties, its cost, and an honest statement of what it would answer if it were run.

Anyone who needs a one-line answer: **ULB and BankSim are measured; the Fraud Detection Handbook is loaded, verified and awaiting a go/no-go on 12 September; AMLSim and PaySim were assessed and rejected for this deadline, for different and specific reasons.**

Dataset	What is in it	Who/what it can link	Status, and why
1. ULB credit card real transactions, anonymised	284,807 transactions, 0.17% fraud (492 cases). Features are anonymised, so no column means anything human-readable.	Nothing. No customer column at all. A “sequence” of 10 rows is 10 different strangers.	MEASURED — 1 condition. The dataset the accepted thesis stands on. Kept deliberately (Decision 1): dropping it would orphan the submitted report, and it is the <i>only</i> reason we found two of our worst bugs.
2. BankSim simulated retail banking	594,643 transactions, 1.211% fraud (7,200 cases), 50 merchants, 180 daily steps.	Customers — 4,112 of them, roughly a hundred transactions each. Time resolves to <i>one day</i> .	MEASURED — 2 conditions. Chosen first because it is one CSV file, small enough to iterate on in minutes, and it directly fixes ULB’s missing-customer problem.
3. Fraud Detection Handbook simulated, ULB research group	1,754,155 transactions — three times BankSim — 0.837% fraud (14,681 cases) over 183 days.	Customers (4,990) and terminals (10,000) , with exact timestamps to one second . The only candidate that links both.	LOADED, NOT RUN — loader landed 8 Sep, 42 acceptance checks pass, zero results . Cost measured at about 11 h of CPU (the original 6 h budget was too low). Go/no-go 12 September .
4. IBM AMLSim simulated multi-bank network	Configurable. Its distinguishing feature is not size but structure: it models <i>several institutions</i> , not one.	Real bank identities — a federation we would not have to invent by dealing customers into piles.	ASSESSED, DEFERRED — right dataset, wrong fortnight. It is a Java generator needing a whole configuration pipeline before a single transaction exists, and the question it was going to settle got settled for free by our control run (Section 7.6). Revisit after the deadline.
5. PaySim simulated mobile money	6.36M transactions with sender, receiver and account balances. Much the biggest of the five.	Little that is useful. No bank identities, and a sending account may appear only once or twice — so there is no history to build a sequence from.	ASSESSED, REJECTED — fails Rule 8. It cannot test the federated story (no banks) and it cannot test the sequence story (no repeat customers). Being the largest file on the list buys nothing if it answers neither open question.

What each one can actually overturn — Size is not the point. **A dataset earns its CPU time by being able to prove one of our claims wrong** (Rule 8), so this is the column that decided the ordering:

- **ULB** can overturn anything that turns out to depend on having customer identities — and it

does exactly that: entity-linked windows help every architecture, which we could only learn by keeping the dataset that has no entities.

- **BankSim** could overturn “our temporal model fails” — and it did, partly. It showed the *windows* were the problem, not the sequence model. But its clock only resolves to one day, so it can only half-answer the timing question.
- **Handbook** is the only one that can finish that job, because its clock resolves to one second and it links terminals as well as customers. Reconnaissance found a fraud-adjacency signal of **71.65**× on terminals, the strongest linkage in the project. It is also the cheapest way to ask the live question: *is Krum’s utility cost a BankSim quirk, or a general property?*
- **AMLSim** was going to be the deciding experiment for that same Krum question, because it has genuine institutional boundaries. **Then our own three-hour control answered the question without it**, and its promotion rationale was spent. We recorded that demotion as a result following the data, not as a convenience.
- **PaySim** can overturn nothing we currently claim. That is the entire verdict.

A sixth file that looked like a dataset and was not

`datasets/bsNET140513.032310.csv` sits on disk next to the BankSim file and looks for all the world like extra data. **It is not.** We checked it row by row: its columns `Source,Target,Weight,typeTrans,fraud` are *byte-identical* to `customer,merchant,amount,category,fraud` in the file we already load, across all 594,643 rows. It is a strict subset of BankSim with four columns deleted, so it carries **zero** information the loaded file does not already have.

It is recorded here because it is the sort of thing that quietly becomes “we used four datasets” in a later draft. **Counting a file twice is not breadth.** The check cost twenty minutes and it is written down so nobody spends that twenty minutes again.

Why the answer is two datasets and not five, stated plainly

It would read better to say we tested on five datasets. Rule 2 does not allow it, so here is the honest version.

Getting a dataset from “interesting” to “a column in our results table” costs a loader, a reconnaissance pass, and then a full re-run of every sweep — **roughly 11 hours of CPU for the Handbook, and that is the cheapest of the three remaining candidates.** With 14 days to a deadline and a repaired hyperparameter search that has to be re-run first, five measured datasets was never on the table.

The choice was therefore between *two datasets with all six claims tested on them*, and *three datasets with one claim tested and five claims left on ULB*. Rule 8 picks the first, because a reviewer attacks the five claims you left behind, not the one you showed off. **So: five datasets assessed, two measured, and this page saying exactly which is which.**

7.4 What the second dataset said about our own detector (31 August)

With BankSim wired in, we could finally ask the question ULB could not answer: **is our detector actually bad, or had it just never been given real sequences to look at?**

To answer it we ran every combination of two things: **seven different detector architectures** (ours, plus six alternatives including the base paper’s) against **two ways of building the 10-transaction window** — either 10 rows straight from the file, which is what ULB forced on us, or 10 transactions belonging to the *same customer*, which is what BankSim makes possible. Three runs each, so the numbers are averages rather than single throws. We also re-implemented the base paper’s four classifiers and four optimisers ourselves on our own data, rather than copying figures out of its tables.

The answers were not comfortable, and they were not all in the same direction.

What we measured	What it means
Giving each architecture <i>customer-linked</i> windows instead of file-order ones improved every single one of the seven, by +0.0999 to +0.1717 MCC.	The window was the problem, not the model. “Temporal complexity does not pay” turns out to have been a statement about windows containing no real sequences. It does not generalise.
On those linked windows our detector ranks last of seven (0.7089) — behind even its own simplified version with the attention removed (0.7733), and behind a plain CNN. The best is a standard LSTM at 0.7805.	... but our architecture is still the wrong one. Fixing the window did not rescue it, and we report that.
On ULB’s file-order windows our detector ranks 5th of seven (0.3296); ResNet-1D leads at 0.5995.	Same conclusion from the other direction. Our detector is a component we measured honestly, not one that wins.
The attention mechanism — the “A” in ADTCN — helps in one setting and hurts in the other : −0.0645 on BankSim linked windows, +0.0502 on ULB file-order windows.	A clean single-factor test, since the comparison model is deliberately just ours with the attention removed. It says the right architecture <i>depends on the setting</i> , which is a more interesting result than either number alone.
On ULB, all five optimisers returned exactly 5.0000 — the highest score the scale allows.	Not a fact about optimisers at all. Five different algorithms agreeing perfectly means something is wrong with the thing they are all being scored by. This one line opened the entire audit in Section 7.9.

How we talk about our own detector and search — a decision, not an accident

Our detector (ADTCN) came last of seven. Our search (DB-BOA) does not beat a setting we picked by hand. It would be easy to bury both. Here is the position we took instead, and why it is defensible.

Neither of them is one of the six promises in the title. Look back at the component table in Section 3: the detector is the foundation the system is built on, and the search is an automation convenience. Neither one has to *win* for the thesis to hold. So we report them as **measured components**: we built it, we measured it honestly, and it did not beat the simple baseline. In a systems paper that is a perfectly legitimate result.

What would not be legitimate is quietly picking whichever dataset or setting makes them look better and reporting only that. So we report every setting we ran, including the ones where we come last.

One more thing we state plainly rather than let a reader assume: **we do not reproduce a single number from the base paper.** Their results are MATLAB runs on their own data. We re-implemented their methods and ran them on *ours*, with the same computational budget cut applied identically to every one of them. Comparing our table against theirs directly would be comparing two different experiments.

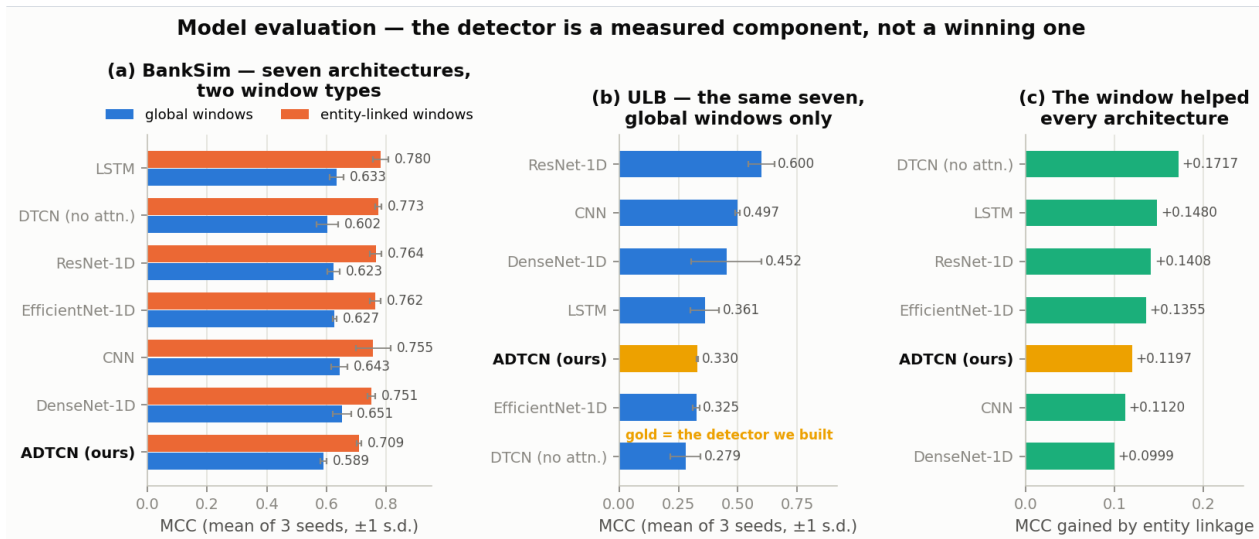


Figure 3: The 2×7 grid, drawn. (a) On BankSim, entity-linked windows beat global windows for *every* one of the seven architectures — and ADTCN comes last of seven on exactly the linked windows it was designed for. (b) On ULB, where there are no customer identities and only global windows exist, the same seven re-rank and ADTCN sits fifth. (c) What the window buys, per architecture: positive everywhere, +0.0999 to +0.1717 MCC. This is why we report *the window*, not the sequence model, as the thing that mattered. Bars are means of three seeds, ± 1 s.d.

7.5 Taking the system claims off one dataset (1 September)

At this point the second dataset had reached the detector and gone no further. Four of the six promises were still resting on ULB alone — not because we had chosen that, but because four of the experiment scripts had the ULB file wired into them and physically could not read anything else (B2).

DECISION 2 — Breadth across *claims* before breadth across *datasets*

Question raised: With one new dataset working, do we add a *third* dataset (which sounds more thorough), or spend the time getting the *existing* claims off ULB?

Decision taken: Get every claim off ULB once, before any claim goes to a third dataset.

Why: A claim tested on three datasets while four sibling claims sit on one is a lopsided table, and a reviewer attacks the four, not the one. This is now Rule 8.

Outcome: A shared dataset-resolution module, all four sweeps taking `--dataset/--partition`, provenance stamped into every result file, and four sweeps re-run on BankSim in 3h15m. **Five of six title claims came off ULB**; the sixth group is dataset-independent by design. Two latent bugs surfaced in the smoke test that would have spoiled the overnight run — and two *worse* ones surfaced by reading the output instead of trusting the exit code: the run was overwriting the original ULB figures in place, and the generated prose asserted ULB conclusions regardless of what had been measured. A draft that contradicts the table printed directly above it is the B9 failure with the numbers left in. **Regenerating any draft now takes seconds and reads from the data.**

7.6 The control that killed our best new claim (1–4 September)

The BankSim runs produced a genuinely exciting finding. Krum — the rule that throws out a sabotaged bank — catches the attacker every single time, but it appeared to **charge us 1.5 to 12.6 percentage points of accuracy** for that protection whenever the banks held completely separate sets of customers. We even had a tidy explanation for why. It was the most quotable sentence we had written all week.

It was also *confounded*, and that word is worth defining because the rest of this section turns on it. A comparison is *confounded* when you change two things at the same time and then credit the result to one of them. Every comparison we had run changed both the **dataset** (ULB → BankSim) *and* the **way customers were dealt out to the three banks** (mixed together → kept separate). So when the number moved, we had no way of knowing which of the two changes moved it — and we had confidently credited the second one.

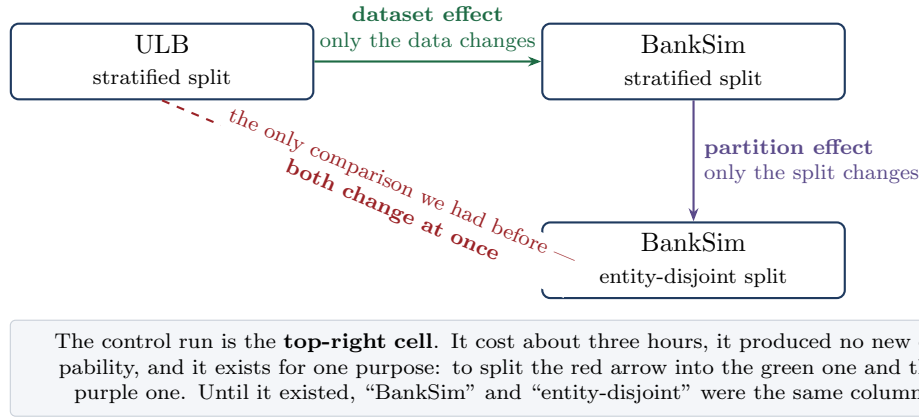


Figure 4: Why the confound control had to exist. Without the top-right cell, “BankSim” and “entity-disjoint” are the same column and no divergence can be attributed to either.

DECISION 3 — Run a three-hour control instead of starting the third dataset

Question raised: We have a strong new finding and a deadline. Do we build on it (third dataset), or spend three hours trying to break it?

Decision taken: Run the control first — BankSim with a *stratified* split, so the dataset changes without the partition changing.

Why: If the finding is wrong, everything built on top of it inherits the error, and a third dataset would have added a third *confounded* column at the same time.

Outcome — the control falsified our own claim. Krum is negative in **7 of 8 cells under the stratified partition too**, where organisations are *not* customer-disjoint. The dataset effect is negative in all 8 rows (−1.06 to −17.27 pp); the partition effect changes sign — and at $n = 7$ it is **positive in all four rows**, the exact opposite of our stated mechanism. **“Krum pays utility under entity-disjointness” is withdrawn.** Two further pre-registrations died the same way: lone-attacker isolation also tracks the dataset (it fires 3/3 on *both* BankSim partitions and 0/3 on ULB), and Monte-Carlo rank fidelity moves +0.783 on the dataset axis against −0.322 on the partition axis. **Every divergence we had attributed to federation structure was actually BankSim.**

The table below is how that was worked out, and it is worth reading slowly. **Each number is “Krum minus FedAvg”** — how many percentage points of balanced accuracy we gained (positive) or lost (negative) by picking the most typical bank instead of averaging all of them. The first three columns are the three conditions we ran. **The last two columns are the answer:** the *dataset* column isolates what changed when only the data changed, and the *partition* column isolates what changed when only the customer split changed. Before the control run, those two columns could not be separated at all.

Krum – FedAvg (pp, balanced acc.)	ULB/strat	BS/strat	BS/entity	dataset	partition
$n=5$ sign-flip	+0.03	−4.62	−11.20	−4.65	−6.58
$n=5$ scaled	+12.49	+0.54	−1.48	−11.95	−2.03
$n=5$ gaussian	−0.01	−1.07	−12.57	−1.06	−11.50
$n=5$ label-flip	+0.04	−5.50	−11.13	−5.54	−5.63
$n=7$ sign-flip	+0.13	−9.15	−4.73	−9.28	+4.43
$n=7$ scaled	+12.48	−4.79	−1.65	−17.27	+3.14
$n=7$ gaussian	+0.00	−11.96	−10.60	−11.96	+1.36
$n=7$ label-flip	+0.00	−11.61	−6.32	−11.61	+5.28

What survived, and it is worth having. The security property is untouched and is now stronger than before: **Krum catches the attacker 8 times out of 8 in all three conditions.** That is the claim Krum is actually there to support, and it now holds on two datasets and three splits.

What we lost is the *explanation* for the accuracy cost, not the cost itself. Krum can cost accuracy; the amount tracks the dataset; and **we do not currently know why.** Writing that down is uncomfortable and it is the correct thing to do — a measured effect with no explanation is a result, while an explanation invented afterwards to cover a measurement is not. Question Q3 in Section 12 carries it as an open problem rather than papering over it.

A trap in the score itself, which we then had to map cell by cell

While reading those results we noticed something impossible: in several runs, the model that had been **attacked** scored *higher* than the same model with no attack at all. An attack cannot genuinely improve a model, so the score was misleading us.

Here is what was happening. Balanced accuracy treats catching fraud and catching non-fraud as equally important. The attacker’s noise happened to push the model toward flagging more transactions as fraud. That caught more of the rare frauds, and balanced accuracy rewarded it — even though the model had become useless in every practical sense. **So the score was crediting the damage.**

We mapped exactly which results this affected, and the pattern is uncomfortable: **0 of 8 cells on ULB, 2 of 8 on BankSim/stratified, and 5 of 8 on BankSim/entity-disjoint** — worst, in other words, precisely in the condition we had built our headline on. Every cell there worse than −5 pp is a contaminated one. Throwing out the contaminated cells, the real cost is −4.73 to −1.48 pp, not the −12.57 to −1.48 we had been quoting.

And the important part: the withdrawal survives the clean-up. On the three cells that are uncontaminated in all three conditions, the same conclusion still holds — so it is a real finding, not an artefact of our own filtering.

7.7 Propagating a withdrawal is its own job (4 September)

A claim is not withdrawn when you decide it is wrong. It is withdrawn when the last script that prints it has stopped printing it. Those are different days, and the gap between them is where retracted numbers come back to life.

So we searched the whole repository for the claim we had just killed, and found it **still alive in two automatically generated drafts**, plus a separate over-claim sitting in the supervisor brief. We fixed all of them *in the scripts that write them*, not in the documents, and regenerated.

That search also turned up a bug worth naming, because it will happen again. **The offending sentence was written by a script that reads only one result file.** Think about what that means: a script looking at a single condition *cannot possibly* tell whether an effect came from the dataset or from the customer split — it has never seen the other conditions. So any explanation it prints is one its own evidence could never contradict. The rule we adopted: **a script may only assert something its own input could have proved wrong.** Explanations now come only from the one script that reads all three conditions together.

One accidental discovery along the way: **the supervisor brief’s PDF had been silently stuck two revisions behind its source for days.** The LaTeX installation was missing a font package, which was quietly aborting every build — so the file on disk kept being the old one. The document the supervisor actually reads had not been rebuilt, and nothing anywhere reported an error. Fixed.

7.8 The privacy budget: we measured it, and then retired it

This one needs a short setup, because the mistake in it is subtle and it is the most instructive thing in the report.

The setup. The banks add noise to protect privacy. Too much noise and the “who contributed most” ranking comes out wrong, so banks get paid the wrong amounts. The dial controlling how much noise is added is ε (epsilon): **small ε means heavy noise and strong privacy; large ε means light noise and weak privacy.** Our contribution is that noising the *published contribution score* survives far heavier noise than noising the *weights*. To put a number on “far”, we defined ε^* : **the heaviest noise setting at which the payments still came out mis-ranked.** Compare the two channels’ ε^* and you get an improvement factor — we had been quoting “60 times better”.

The problem with that number (B7). We had only tested up to $\varepsilon = 3000$, and the weight channel was *still failing* at the very top of that range. So the true crossover was somewhere beyond where we had bothered to look. What we had was a floor — “at least 60 times” — and we had printed it as a measurement. This was the only open item that could actually *un-withdraw* a claim, and it was the cheapest thing on the board to fix, so we fixed it.

DECISION 4 — Extend the grid *and* the draw count, on one shared grid for all three conditions

Question raised: Extend the grid (test further out) or raise the repeat count (measure the rate more precisely)? They fix different things.

Decision taken: Both, and **every condition re-run on the same extended grid** — 11 points out to $\varepsilon = 30000$, then a second pass at 1000 draws on the budgets that actually decide each threshold. Full scope approved, about 4 hours of CPU.

Why: Three columns on three different grids are not comparable. And a correction found *before* the run showed the two knobs are not interchangeable: more budgets test further out; more draws tighten the estimate at a fixed budget.

Outcome: The stated goal was met — **all three conditions de-censored**, every factor became a measurement rather than a lower bound. **And then the measurement turned out to be the wrong thing to quote.**

The correction that reframed the whole objective — and it came from reading our own code, not from a result

Then, reading the code rather than the output, we realised something that invalidates the number entirely.

ε^* is defined as *the largest noise setting at which we caught a mis-ranking*. Look carefully at what that definition does when you test harder:

- **Run more trials at each setting** and you can only ever *find* more failures, never fewer — the extra trials are added to the old ones, they do not replace them.
- **Test further out** and you can only ever add new candidates for the maximum, never remove existing ones.

So neither knob can ever push ε^* down. It can only go up, or stay put, forever. Which means our number was never converging on a true value at all — it was simply recording *how hard we had looked so far*. Read it correctly, ε^* means “the largest setting at which we happened to catch a failure”, and it gets bigger every time anyone tries harder. That is a fact about our measuring procedure, not about the system we were measuring.

And then the same day demonstrated it to us twice, in the most direct way possible. After the first pass, BankSim/entity-disjoint looked like our best-supported column and we wrote down that it should be preferred over ULB. **Six hours later, looking ten times harder found exactly one failure in a thousand trials** at the top of its range. That single failure pushed its threshold up, pushed its factor from $33\times$ to “at least $100\times$ ”, and turned the number straight back into a floor. The ranking of which condition to trust flipped back to where it had started — **on the strength of one event in a thousand**.

The same data, read at three different rate thresholds	rate > 0	rate > 0.01	rate > 0.05
ULB / stratified	$60\times$	$20\times$	$33\times$
BankSim / stratified	$30\times$	$60\times$	$20\times$
BankSim / entity-disjoint	$\geq 100\times$	$33\times$	$33\times$

The factor swings from $20\times$ to $\geq 100\times$ on an arbitrary cut-off, **and the ranking of the three conditions changes with it**. Only the leftmost column was ever published.

DECISION 5 — Retire the improvement factor as a headline

Question raised: We finally have measured numbers. Do we publish the factor?

Decision taken: No. **Retire it**, and carry the contribution on two threshold-free statements instead.

Why: Rule 4. Picking whichever cut-off flatters the mechanism is result-shopping, and no single number from this family is defensible. The mechanism does not need it.

Outcome — what carries the claim now:

- (1) **The ordering.** The output channel mis-ranks rewards no more often than the weight channel — 12 of 12 probed budgets, and 11/11, 11/11, 10/11 across the full grid. It is *paired within each draw*, so sample size changes its precision, never its direction. It has now survived three conditions, two sample sizes and two metrics.
- (2) **The rank-fidelity gap at a stated budget**, which needs no threshold at all. At $\varepsilon = 50$: $\rho = -0.199 \rightarrow +0.995$ (ULB), $+0.017 \rightarrow +0.945$ (BankSim/stratified), $-0.070 \rightarrow +0.805$ (BankSim/entity-disjoint). Same direction in all three.

And one genuinely reassuring result: the four fragile thresholds that had rested on a single inverted draw out of 100 became 7, 11, 16 and 10 per 1000 — all consistent with a true rate near 0.01. **The fragility was about precision, never about existence.**

The cost nobody was auditing: a ULB result no longer reproduces

While verifying that the longer run contained the shorter one, we re-ran the nine historical budgets at identical seeds. **Both BankSim conditions reproduced 9 of 9 bitwise. ULB moved in 13 of 18 cells**, and its ground-truth contribution split moved with them. ULB *does* reproduce itself same-day (11 of 11), so it is deterministic under fixed conditions — the 30 August divergence is a change of *conditions*. We then eliminated everything reachable: every code change on the ULB path was audited and shown provably inert for ULB; every configuration dictionary was compared literal-by-literal; thread-count sensitivity was tested directly (all six weight tensors bit-identical). What remains is the torch/numpy version at the time — **and that is unrecoverable, because no stored result recorded the stack it ran under**.

We recorded it as unexplained rather than guessing a cause. The durable fix shipped the same day: torch version, thread count, numpy version, Python version and platform are now written into *every* sweep result file. *That diagnosis took a day and produced no number; the guard is what stops the next one costing the same.*

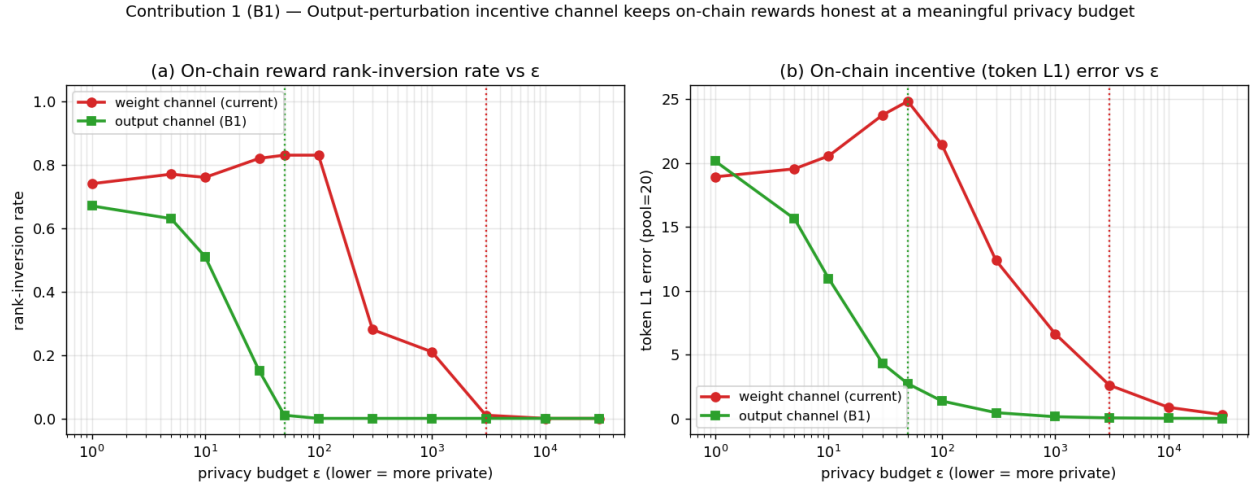


Figure 5: The contribution, on the extended 11-point grid. **(a)** How often the on-chain reward ranking is wrong, against the privacy budget; **(b)** the resulting token error. The green (output) channel reaches faithful rewards roughly an order of magnitude earlier in ϵ than the red (weight) channel. **It is the separation between the curves that is the claim, not the ratio of the two dotted lines** — Section 7.8 explains why the ratio is not a stable statistic.

7.9 The search the framework is named after (4–5 September)

The last and largest piece of work in the period. It started with the loose thread from B5 — five completely different search algorithms all reporting a score of exactly 5.0000 — and it ended somewhere considerably worse than we had expected.

First, what DB-BOA is supposed to do — The detector has settings that have to be chosen: how many filters, how many training steps, and so on. Trying every combination on the real model would take weeks, so DB-BOA does what every hyperparameter search does — it trains a small, fast stand-in model (a **surrogate**) for each candidate setting, scores it, and uses those scores to decide where to look next. The whole method rests on one assumption: **that the score is a fair reflection of the setting**.

The diagnosis — It was not. The scoring function **picked a fresh random sample of rows and a fresh random seed every single time it was called**. So asking it to score the same setting twice gave two different answers. Three measurements pinned down how bad this was:

- **The same setting, scored 25 times, ranged from 3.4497 to 5.0000** — and hit the theoretical maximum once, purely by luck, with no searching involved at all. That alone explains the five identical 5.0000s.
- **Changing the random sample moved the score about as much as changing the model did** (1.0471 versus 1.0794). In other words, roughly half of what the search was “learning” was which candidates got lucky draws.
- **One of the two things it was searching over did nothing at all**. A formula meant to pick a batch size, $\max(32, 1400 // \text{spe})$, worked out to 32 for *every single value* in the range being searched. So the advertised two-dimensional search was really one-dimensional, and every “optimal” value we had reported for that second setting was arbitrary.

Note which way this cuts. It would be easy to read “the search was broken” as an excuse — *our method never really got a fair trial*. It is the opposite. A search that cannot reliably rank its own candidates is effectively returning one at random, and **a randomly chosen setting tying with a sensibly hand-picked one is exactly what you would expect**. The broken search does not

rescue the negative result; it explains it.

Then the shipped path was checked, and it was worse —

We had audited a script — but not the code that actually runs

Our diagnostic script built its stand-in model from a pool of 6,000 rows. **The real pipeline uses a different function, and that one returns 3,000.** We had spent three days auditing something adjacent to the thing that ships.

Follow the arithmetic on the real one. ULB is 0.173% fraud, so a 3,000-row pool contains about **5 fraud transactions**. The stand-in model demands at least 30 to train on. It cannot conjure 25 more, so it drew 30 “with replacement” — meaning it took the same 5 transactions over and over, about six times each. Then it split those 30 rows into 21 for training and 9 for testing. Since all 30 were copies of the same 5, **all 9 test rows were duplicates of rows it had just trained on.**

The model was being tested on its own training data. It scored perfectly. **The 5.0000 was not luck — it was memorisation**, and it would have happened every time.

Two things make this worse. First, **the repair we had already designed would not have fixed it**: our plan was to force the split to include fraud on both sides, which with only 5 unique transactions is precisely the mechanism that puts copies on both sides. Second, BankSim was completely clean throughout (38 and 75 unique frauds), so **a project using only the second dataset would never have seen this.** The rule we took away: **audit the code path that ships, not one that resembles it.**

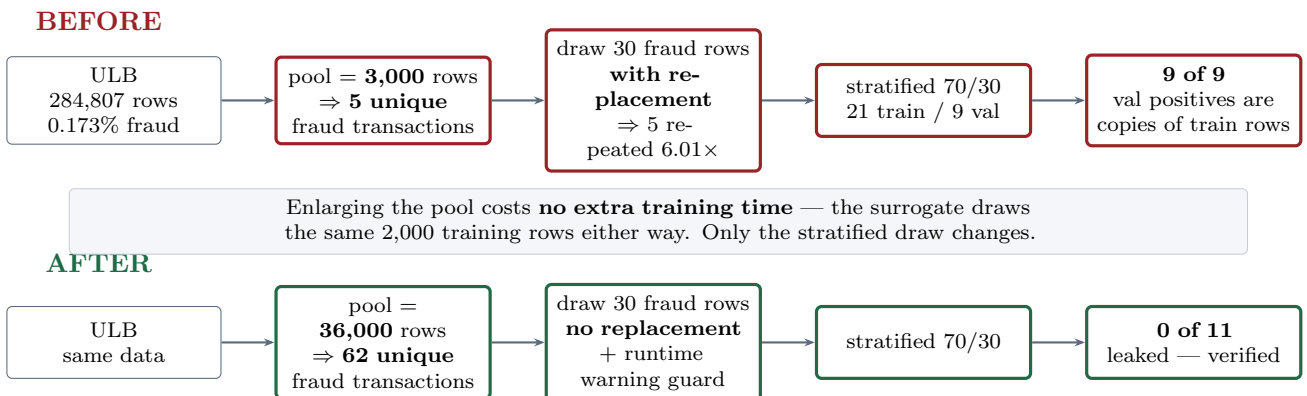


Figure 6: The leak, and the fix. The defect was in the *pool*, one layer below where the planned repair operated, which is why the repair could not reach it.

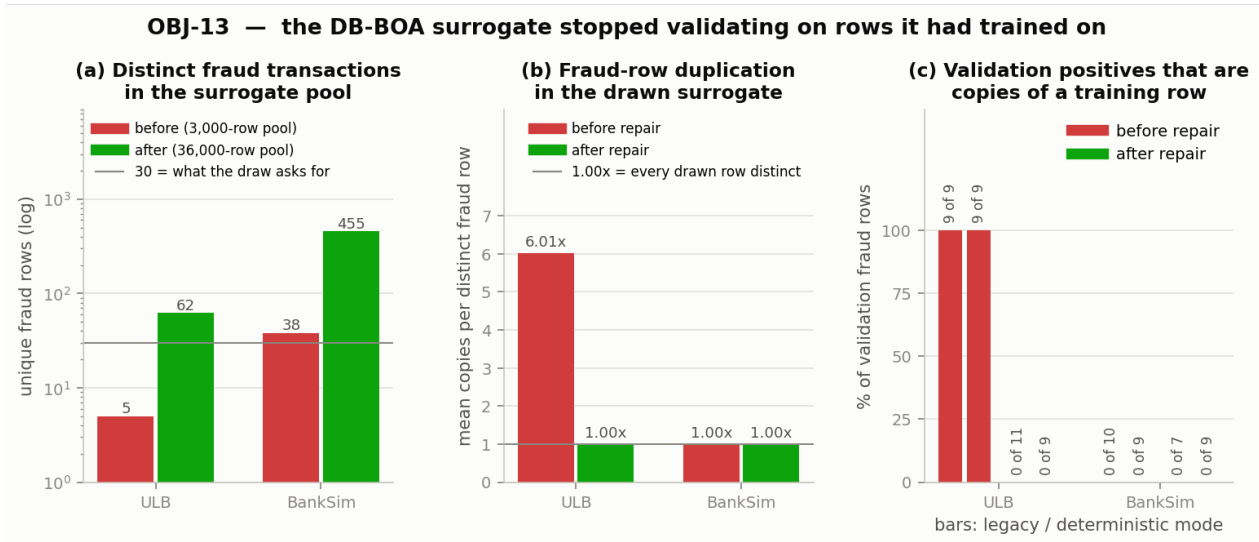


Figure 7: The same repair, measured rather than drawn — the memorisation-to-learning transition, before and after. **(a)** The 3,000-row pool held *five* distinct fraud transactions on ULB against a draw that asks for thirty; the 36,000-row pool holds 62. **(b)** With five available, the draw took each one about six times over; afterwards every drawn row is distinct on both datasets. **(c)** The consequence, and the defect itself: before the repair *every* ULB validation positive was a copy of a row in the training half, in both surrogate modes — the 5.0000 ceiling was memorisation, not luck. BankSim was clean throughout, in both states, which is precisely why keeping the second dataset is what exposed the bug. Read from the pre- and post-repair runs of `obj13_shipped_path_check.py`.

DECISION 6 — Fix the pool by enlarging it, not by re-weighting or clamping

Question raised: Three ways to stop the surrogate validating on memorised rows. (a) Enlarge the pool so 30 genuinely distinct fraud rows exist at the real rate. (b) Oversample fraud *into* the pool. (c) Never sample with replacement — accept whatever few unique frauds exist.

Decision taken: (a). Pool raised from 3,000 to 36,000 rows for both datasets, plus a runtime warning that fires if the replacement branch is ever taken again.

Why: (b) would make the surrogate see a fraud rate that is not the dataset’s — the exact defect the function was written to avoid. (c) would leave ULB’s surrogate training on **five** fraud rows, which is honest and useless. Only (a) keeps the rate real *and* the positives distinct. The warning was added because **the cost of this defect was its silence**: a config value alone would fix today’s datasets and let the next one reintroduce it without a sound.

Outcome: Verified 0 of 11 validation positives leaked, on both datasets, in both modes.

The repair, and three defects it would have shipped with — Fixing the scoring function turned out to be the easy part. Applying that fix turned up **three separate ways the repair would have quietly destroyed the evidence for its own necessity** — all three found by checking the output rather than trusting the exit code:

1. The patch **silently re-pointed the diagnostic script** — the one that produced this entire objective’s evidence base — at the new default mode. It would have reported a within-seed standard deviation of **exactly 0.0000** and read as “*the objective was always fine*”, which is the precise opposite of the finding. It now pins the old behaviour explicitly, with the reason written at every call site so it is not “simplified” away later.
2. The same silent switch in the base-paper comparison script, whose stored results *are* the 5.0000 evidence. It now stamps the mode into its own output and **refuses to overwrite** a file recorded under a different one. A warning would not do — a warning scrolls off the top of a six-hour run.

3. A pre-existing filename collision in which two separate result tracks overwrote each other, leaving only whichever ran last. **This was the third instance of the same class of bug** in a fortnight: an artefact whose only copy lives somewhere the next run can destroy.

We also verified the old behaviour is reproduced **bit-for-bit** — three configurations agreeing to ten decimal places against a saved copy of the pre-patch module — so the comparison runs against a live baseline rather than a remembered one.

DECISION 7 — Run the comparison at full deployment budget, against the recommendation

Question raised: The side-by-side could run cheaply (about 1.1 h, matched to the base-paper budget) or at the *deployed* search budget (about 16.7 h). The written recommendation was the cheap option.

Decision taken: **The expensive one** — population 20×30 , three surrogate protocols $\times$ three seeds, BankSim only, run detached overnight.

Why: Only the deployed budget yields configurations comparable to the one actually shipped in our results (142 filters / 76 steps-per-epoch). A cheaper run would have answered a question we were not asking. BankSim only, because at that moment ULB’s pool did not describe its own deployment, so a ULB arm would have cost 17 hours to produce a number needing a caveat longer than the result.

Outcome: 16.3 hours, 9 of 9 searches completed, clean exit. **Not one of the nine searched configurations beats a hand-set 128 filters / 150 steps-per-epoch.** If this run ever has to be repeated, repeat it at the same budget — do not “optimise” it back down.

Surrogate mode	Filters chosen	paired Δ vs default	wins/7	Reading
legacy (unrepaired)	81 / 69 / 135	−0.2027 / −0.1130 / −0.1780	0 / 1 / 1	every seed below the default
deterministic	137 / 56 / 62	−0.1654 / −0.1647 / −0.0720	0 / 2 / 4	holds the best single search
averaged (shared draws)	137 / 146 / 149	−0.1654 / −0.3512 / −0.3716	0 / 1 / 1	converges, and generalises worst
<i>the configuration we shipped</i>	142 filters, 76 spe	−0.1900	2	also loses to the default

Objective seeds are 42 / 7 / 123 in every row. Mean paired difference across all nine: **−0.1982**. Compared **paired** on seven shared held-out draws, because the yardstick uses common random numbers — judging these by the ± 0.4 spreads on the individual means would call a real difference “noise”.

Two findings that inverted what we had predicted —

1. What fixes a noisy search is not freezing the randomness — it is sharing it

This one surprised us completely, and it is the most useful thing in the report for anyone doing similar work.

There are two obvious ways to stop random scoring from wrecking a search. **(a) Freeze the randomness:** use the same fixed seed every time, so the scoring function stops being random. **(b) Share the randomness:** let it be random, but score *every candidate on the same random sample*, so they are all judged on the same test. We expected (a) to be the fix, and we had predicted that neither would fully work.

Look at how widely each mode’s chosen answer scattered across three runs (the setting ranges from 5 to 255): **legacy** (broken) scattered by **66**. Merely freezing the seed — **deterministic** — scattered by **81**, which is *worse than doing nothing*. Sharing the sample — **averaged** —

scattered by **12**.

Freezing the randomness achieved nothing. Sharing it collapsed the scatter almost to zero. The reason is simple in hindsight: a frozen seed still gives every candidate a *different* sample, so the comparison between them is still corrupted. What matters is not that the noise is repeatable, but that all candidates face the identical test. That detail was added to our code almost as an implementation convenience, and it turns out to be the half of the repair that does all the work.

2. The mode that works best at searching is the one that generalises *worst*

Now the sting. Having fixed the search so it finally converges, we scored all three modes on fresh data they had never seen. The winner should have been the fixed one. It came last: **deterministic 3.5857, legacy (broken) 3.5552, averaged (fixed) 3.4237**.

The explanation is straightforward once you see it. Sharing the same three samples across all candidates is what let the search rank them reliably — but it also means the search can now relentlessly optimise *for those three particular samples*. And it did: it converged on high-filter settings (137, 146, 149) that fit those samples beautifully and generalise worse. **Making the scoring function rankable made the search much better at exploiting it.**

The random noise in the broken version had been acting as an accidental brake on overfitting. That is a real and slightly funny finding, and it is emphatically **not** an argument for keeping the broken version — a brake that works by accident is not a brake you can rely on.

And one defect the repair itself revealed — Our repair only touched the stand-in model. The real deployed model still uses the old formula — and here is the twist: on the real model, which trains on roughly 400,000 rows rather than 2,000, that formula does *not* collapse to a constant. **The second setting was dead in the search but alive in the deployed model.**

That is worse than simply wasting one of the two search dimensions. It means the search was blind to a setting the real model genuinely responds to — it could not have found a good value for it even in principle. Whether to change the deployed formula is **deliberately still an open question**, because changing it would invalidate every model we have trained so far, and that is not a decision to take quietly at the end of a section.

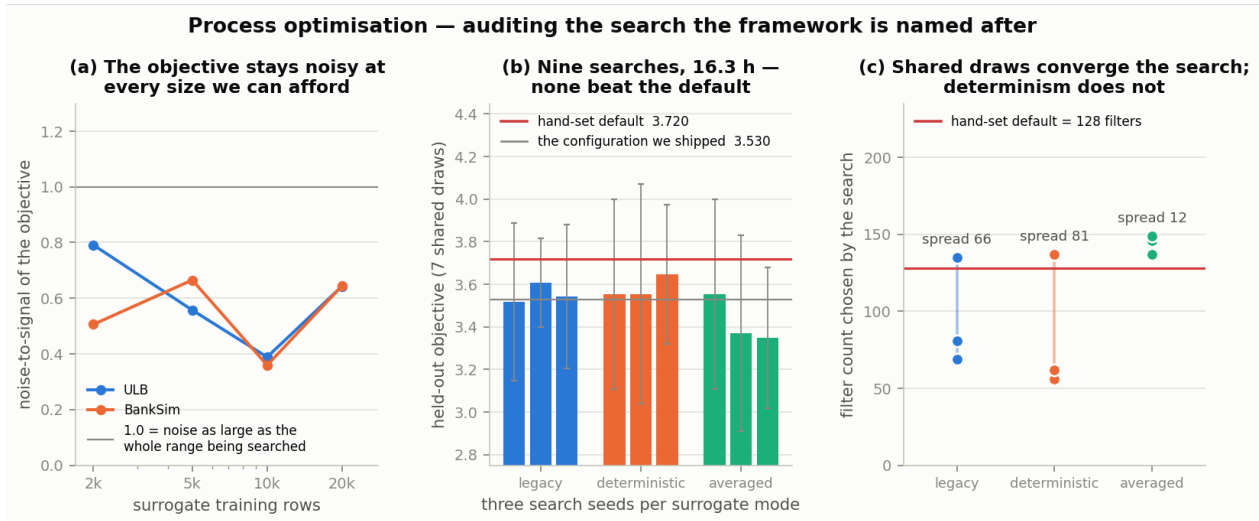


Figure 8: The search the framework is named after, audited. **(a)** The scoring function’s run-to-run noise, divided by the whole configuration-to-configuration range it is meant to rank, at four surrogate sizes on both datasets: it never falls below 0.36, so at every size we can afford, a large part of what the search is ranking is draw luck. **(b)** The nine full searches of the 16.3-hour run, scored on seven *shared* held-out draws: not one of the nine clears the hand-set default (red). **(c)** What each search chose. The mode that shares its random draws across candidates (*averaged*) collapses the spread to 12 filters; the merely *deterministic* mode is the widest of the three at 81. Sharing draws — not fixing the seed — is what converges a noisy search.

The verdict, and why it is the right outcome

The repair works, changes which configuration the search picks, and still does not beat a hand-set default. Rule 3 governs: this was *diagnosis of a negative result, not a rescue attempt*, and it was pre-registered as such before any repaired run. The claim “DB-BOA buys no measured accuracy gain over the hand-tuned default” no longer rests on a statistical tie at $p = 0.34$ — it rests on a direct paired measurement, under three surrogate protocols, at three seeds, at the deployed budget. **The negative result got stronger by being attacked.**

8. The decision log

Every fork in the road during these six days, on one page, with what each choice actually produced. **If you read only one section of this report, read this one.** The pattern worth noticing is that the choices which produced the most value were mostly the ones that went looking for trouble.

	Question	Answer chosen, and why	What it produced
1	Replace ULB with an entity-linked dataset, or keep both?	Keep ULB, add BankSim. A claim is only shown dataset-dependent if both are measured; dropping the losing dataset is result-shopping.	Entity-linkage priced at +0.10 to +0.17 MCC; DP collapse replicated with a <i>flipped direction</i> ; and it exposed two ULB-only defects (the memorised-rows leak, the reproduction failure) that would otherwise still be hidden.
2	Third dataset, or get existing claims off ULB?	Claims first (Rule 8). A lopsided table is not rigour.	Five of six title claims off ULB in 3 h 15 m; generic plumbing so a third dataset is loader work only.
3	Build on the new Krum finding, or spend 3 h trying to break it?	Break it. A control that can overturn a finding outranks a dataset that can add one (now Rule 12).	Our best new claim died , along with two others — all three were attributions to federation structure that belonged to the dataset. Caught by us, not by a reviewer.

	Question	Answer chosen, and why	What it produced
4	Extend the privacy grid, raise the draw count, or both?	Both, on one shared grid. They fix different things and three columns on three grids are not comparable.	All three conditions de-censored; the four fragile thresholds shown to be real; and the estimator shown to be unstable.
5	We finally have measured factors — publish them?	No, retire the factor. It swings $20\times$ to $\geq 100\times$ on an arbitrary cut-off and the ranking changes with it.	The contribution now rests on two threshold-free statements: the ordering (paired, 12/12) and the rank-fidelity gap at a stated budget.
6	Fix the surrogate pool by enlarging, re-weighting, or clamping?	Enlarge (3,000 $\rightarrow$ 36,000) plus a runtime guard. Only this keeps the fraud rate real <i>and</i> the positives distinct.	0 of 11 leaked, verified; and it costs no extra training time.
7	Cheap 1.1 h comparison or the 16.7 h deployment budget?	The expensive one , against the written recommendation — only it yields configurations comparable to what we shipped.	16.3 h, 9/9 clean, 0 of 9 beat the hand-set default ; two predictions inverted; the negative result strengthened.
8	Repair the dead search axis, or drop it?	Repair it. If it turns out not to matter once genuinely alive, that is a <i>measured</i> finding; dropping it now asserts the same conclusion without evidence.	The axis is alive but coarse (23 distinct batch sizes across 201 values), and repairing it revealed the deployed model’s axis was never dead — only the surrogate’s.
9	The 30 August ULB result does not reproduce. Name a cause?	Record it as unexplained. Guessing the torch version would be inventing a cause.	Two ULB files marked not-quotable, and environment stamping now shipped in every sweep result file.
10	Consolidate or expand, with 15 days left?	Consolidate first , then expand; writing block protected.	The surrogate repair had to precede any third dataset, or that dataset’s search arm would be generated on a search we already knew was broken — and paid for twice.
11	Commit the extension, or keep iterating?	Commit. About 20 h of results existed only as untracked files.	Two commits on a branch; and a <code>.gitignore</code> rule written for LaTeX was found to be silently swallowing <i>experiment run logs</i> .

9. Where the six claims stand now

This is the scoreboard. For each of the six promises in the thesis title: which datasets it has now been tested on, and what the evidence says after the period. **Bold text marks what changed in these six days.**

The promise	Tested on	Where it stands now
Secure (the privacy half)	2 datasets, 3 conditions	Holds — and the second dataset revealed something new. Adding privacy noise to the weights destroys the model everywhere (MCC drops to roughly 0). But <i>how</i> it fails differs: on ULB the wrecked model flags nothing, and plain accuracy records the disaster as a fall of 0.11 pp. On BankSim it flags everything — 119,784 false alarms — and accuracy falls 97.55 pp. Same failure, one dataset reporting it as harmless. That is an argument for our choice of score as much as for the privacy claim itself.
Secure (the attack half)	2 datasets, 3 conditions	The security property holds everywhere: the attacker is caught 8 times out of 8 in all three conditions. What changed is the explanation for its accuracy cost. Krum can cost accuracy, the amount tracks the <i>dataset</i> , and our explanation for why is withdrawn with nothing yet in its place.
Incentivized (catching cheats)	2 datasets, 3 conditions	Holds three times over. Two banks colluding are caught in every condition (+40.78/ + 44.39/ + 47.40 and +79.52/ + 86.95/ + 89.73 on the two attack types). But the lone-attacker story changed: the isolation mechanism now <i>fires</i> on BankSim (3 of 3, versus 0 of 3 on ULB) while improving accuracy in 0 of 3 cases in every condition. It goes off, and it does not help. That negative result is now solid across three conditions.
Incentivized (privacy vs rewards)	2 datasets, 3 conditions	The core contribution holds everywhere. Noising the published contribution beats noising the weights at 12 out of 12 tested noise levels, and 11/11, 11/11, 10/11 across the full range — the one exception being the most extreme setting, where <i>both</i> approaches fail almost completely. The “improvement factor” is retired as a headline; the ranking gap at a stated noise level carries the claim instead.
Scalable (fair attribution)	2 datasets, 3 conditions	The exact fairness calculation is expensive, and that cost is structural rather than dataset-specific (113.5 / 140.3 / 120.6s for 12 banks over the same 4,095 combinations) — confirmation, not news. “The fast approximation fails from about 6 banks onward” is withdrawn: the failures are intermittent (5/10, 5/10, 3/10) with no threshold at all. Worse, below about 10 banks the “fast” version is barely faster, and on ULB with 5 banks it is <i>slower</i> than doing it exactly.
RL / Consensus / Blockchain	does not depend on the dataset	Leader election spreads the work far more evenly (inequality falls from 0.90 to 0.782 over 5 runs) and largely stops picking unhealthy nodes (100% → 15.2%). The real Fabric network commits in 2115.9ms on average and peaks at 40.3 TPS, with performance collapsing immediately after that peak.

10. The comparisons, side by side

The earlier sections tell the story in order. This section is for the reader who wants to see the comparisons themselves, laid out rather than narrated — every condition scored on every metric, plus the four things a reviewer normally asks to see drawn: what changed between datasets, the full score table, the statistical tests behind the two negative results, and the measured system performance.

Nothing in this section is new evidence. Every panel is drawn by one script — `experiments/make_report_figures.py` — reading the same stored result files quoted earlier. That script does no training and no sampling of its own, so it cannot introduce a number that was not already measured.

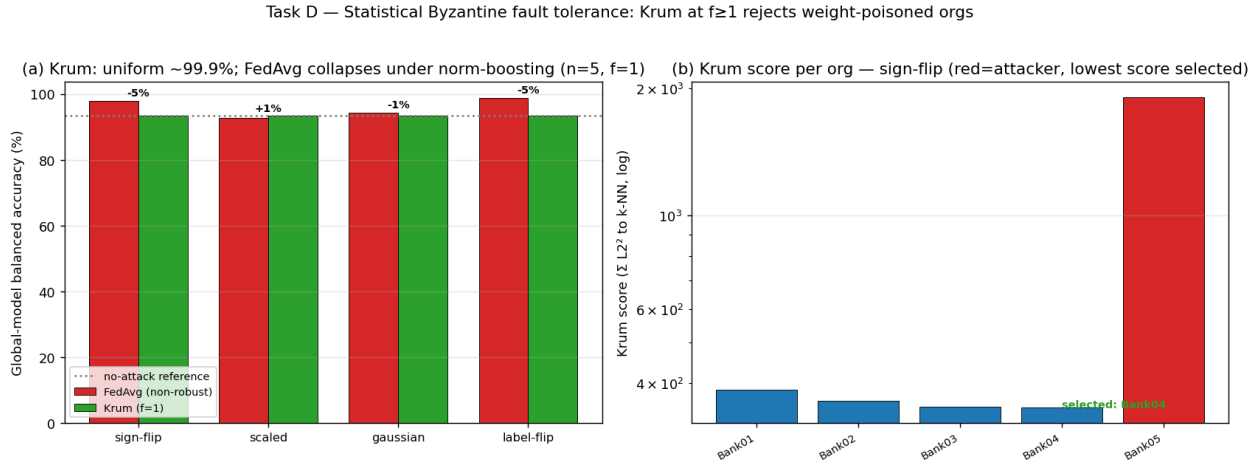


Figure 9: The Byzantine layer on the control condition. **(b)** shows the security property that replicates everywhere — the poisoned organisation’s Krum score is orders of magnitude above the others, so it is never selected. **(a)** shows the part we had to withdraw a mechanism for: the accuracy difference between Krum and unprotected averaging is small and mostly negative here, and it tracks the dataset rather than the federation structure.

Count the columns before you read them

Every figure in this section has **three** columns, and it is worth being clear about why, because Section 7.3 compares *five* datasets.

The three columns are **ULB/stratified**, **BankSim/stratified** and **BankSim/entity-disjoint**. That is **two datasets in three conditions**, not three datasets. The third column is a *control* — it exists purely to separate “the data changed” from “the customer split changed”, which is the whole subject of Section 7.6.

Of the other three datasets on the five-way comparison: the Handbook is loaded and verified but has not been run (a go/no-go on 12 September), and AMLSim and PaySim were assessed and not run at all. Under Rule 2 an unrun dataset gets no column, so it has none here. **Every figure below has three columns because three is exactly what was measured.**

10.1 What changed when the dataset changed

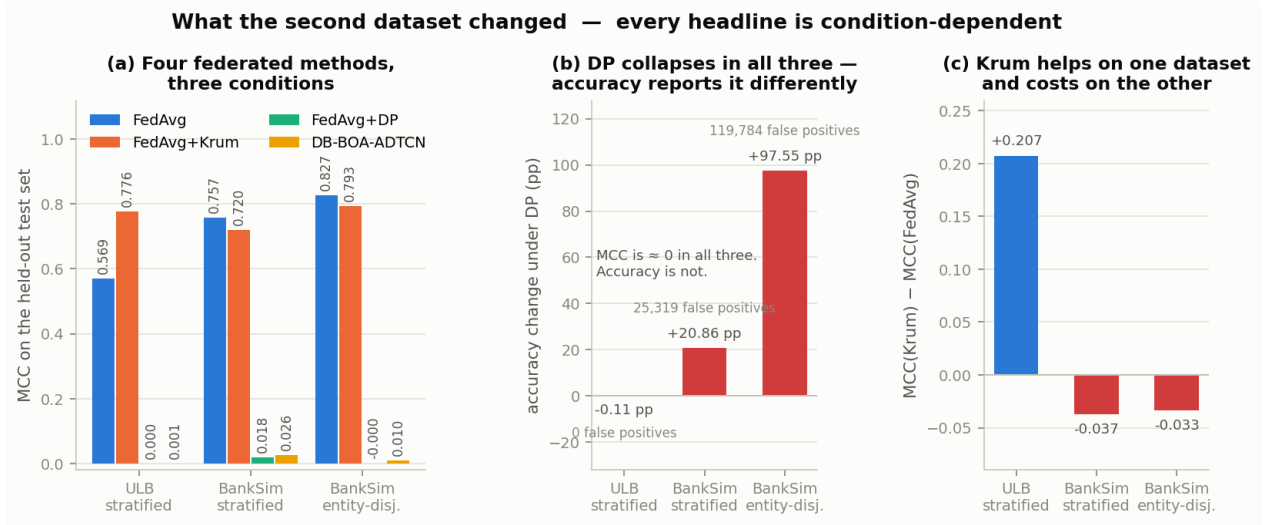


Figure 10: The three conditions, compared on the things that moved. **(a)** The same four federated methods scored under each condition: the ordering of FedAvg and FedAvg+Krum *swaps* between ULB and BankSim, and the DP arm is at $MCC \approx 0$ in all three. **(b)** Why the metric choice is part of the claim: differential privacy destroys the model in every condition, but accuracy reports that as -0.11 pp on ULB and $+97.55$ pp on BankSim/entity-disjoint — the same failure, read as almost harmless on the 0.17%-fraud file. **(c)** What Krum costs: positive on ULB, negative on both BankSim conditions, tracking the *dataset* rather than the federation structure. That is the measurement that killed our best new claim.

10.2 The score index — every metric, every condition

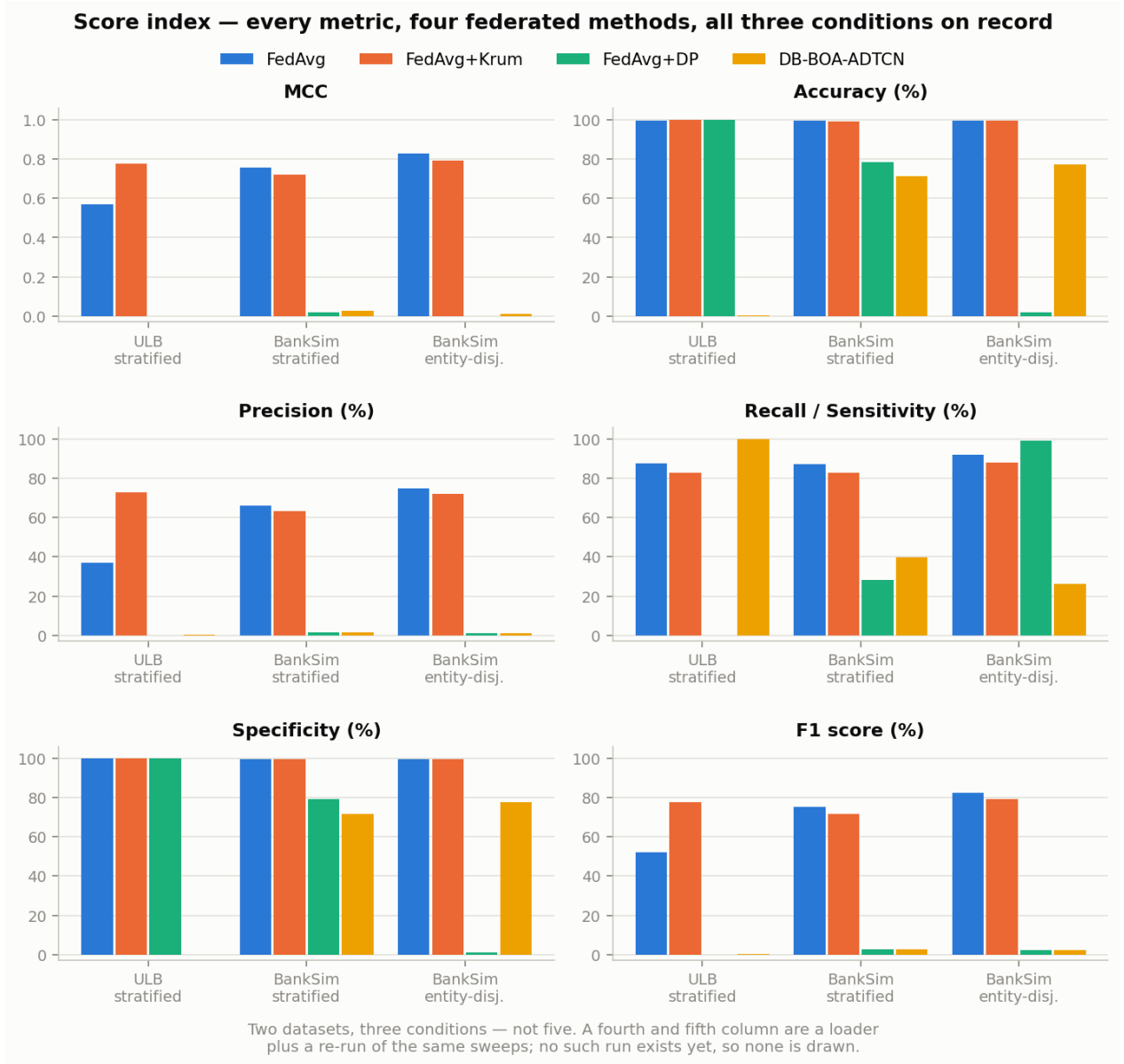


Figure 11: The full score index as small multiples: six metrics, four federated methods, three conditions. The panels are worth reading against each other rather than one at a time — accuracy separates the methods least and MCC separates them most, and the DP arm is the case that proves it.

Method	MCC	Accuracy	Precision	Recall	Specificity	NPV	F1	FPR	TP	FP	FN
<i>ULB / stratified</i>											
FedAvg	0.569	99.72	37.07	87.76	99.74	99.98	52.12	0.26	86	146	12
FedAvg+Krum	0.776	99.92	72.97	82.65	99.95	99.97	77.51	0.05	81	30	17
FedAvg+DP	0.000	99.83	0.00	0.00	100.00	99.83	0.00	0.00	0	0	98
DB-BOA-ADTCN	0.001	0.21	0.17	100.00	0.04	100.00	0.34	99.96	98	56843	0
<i>BankSim / stratified</i>											
FedAvg	0.757	99.38	66.23	87.20	99.51	99.86	75.28	0.49	1151	587	169
FedAvg+Krum	0.720	99.29	63.23	82.73	99.48	99.81	71.68	0.52	1092	635	228
FedAvg+DP	0.018	78.52	1.44	28.03	79.07	99.02	2.74	20.93	370	25319	950
DB-BOA-ADTCN	0.026	71.29	1.50	39.62	71.64	99.09	2.89	28.36	523	34307	797
<i>BankSim / entity-disjoint</i>											
FedAvg	0.827	99.58	74.81	91.82	99.66	99.91	82.45	0.34	1212	408	108
FedAvg+Krum	0.793	99.50	72.05	87.88	99.63	99.87	79.18	0.37	1160	450	160
FedAvg+DP	-0.000	2.03	1.08	99.02	0.97	98.90	2.14	99.03	1307	119784	13
DB-BOA-ADTCN	0.010	77.07	1.27	26.36	77.63	98.98	2.42	22.37	348	27063	972

Every number above is read from the three `baselines*.json` files, one per condition; the machine-readable form is `results/figures/score_index.csv`. DP runs at $\varepsilon = 1$, $\delta = 10^{-5}$ throughout. The DB-BOA-ADTCN row on ULB is a degenerate all-positive model — that is why its accuracy is 0.21% and its recall 100%. It is reported, not hidden.

10.3 The two negative results, tested

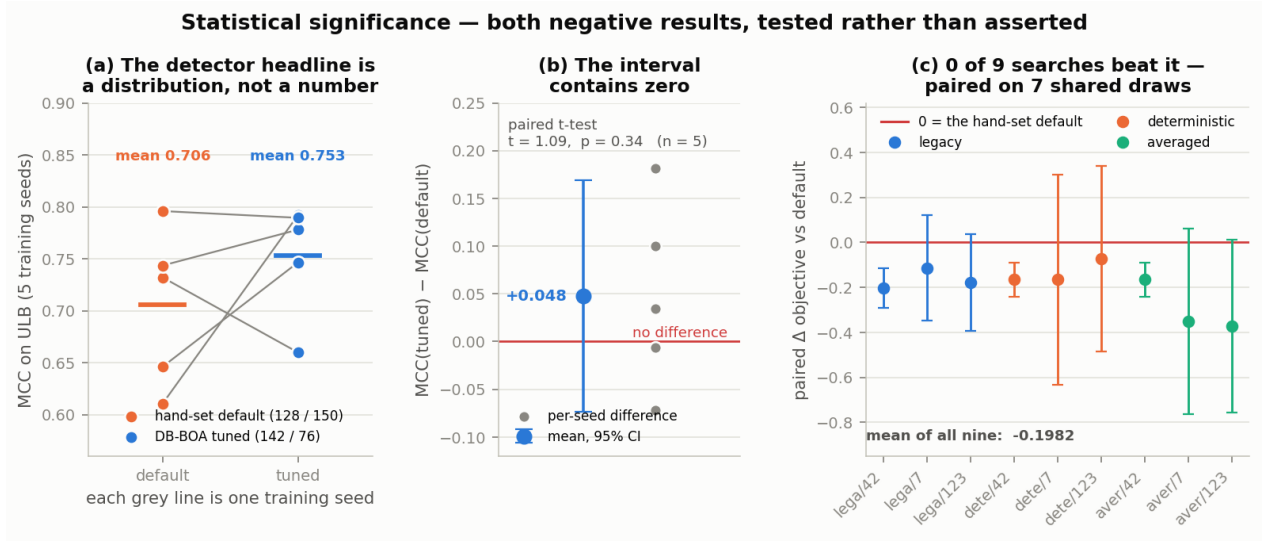


Figure 12: Both negative results carry a test, not an assertion. **(a)** The detector headline over five training seeds: the tuned and default configurations interleave, and two seeds put the default above the tuned one. **(b)** The paired difference is +0.048 with a 95% interval of $[-0.074, +0.169]$ — it contains zero ($t = 1.09$, $p = 0.34$, $n = 5$). **(c)** The stronger version of the same result, from the 16.3-hour run: each of the nine searches differenced against the hand-set default *on the same seven held-out draws*, so draw luck cancels. All nine means are below zero; the mean of the nine is -0.1982 . Paired on shared draws is the honest comparison here; the ± 0.4 spread of the individual means would call a real difference “noise”.

10.4 The system, measured

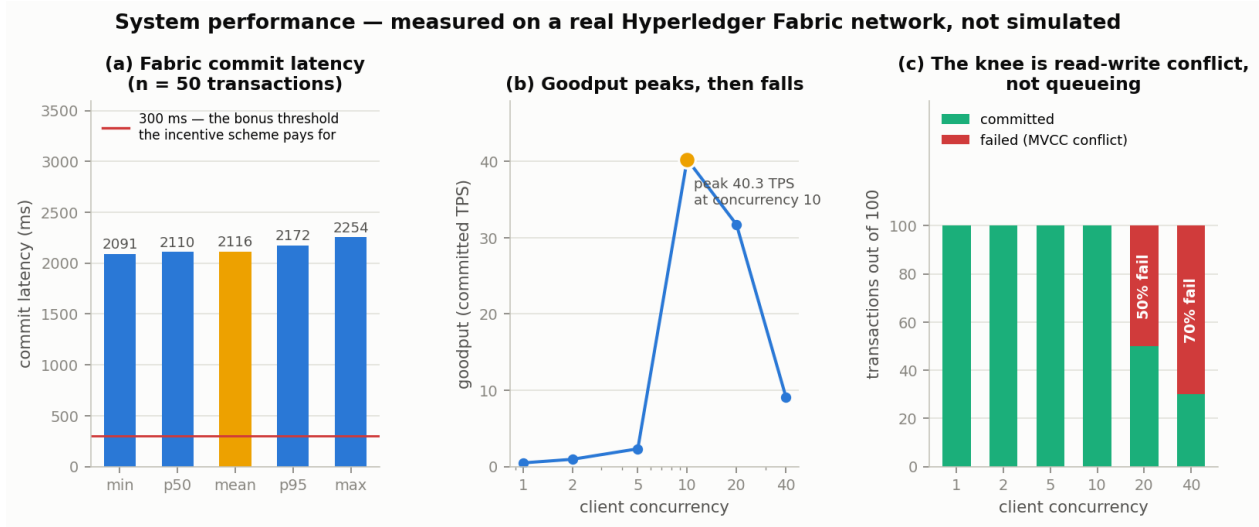


Figure 13: The blockchain layer, on a real Hyperledger Fabric network. **(a)** Commit latency over 50 transactions is tightly clustered around 2.1s — and the incentive scheme’s +15-token bonus for committing below 300 ms (red) is unreachable on a Raft floor an order of magnitude above it. That design mismatch was found by measuring, not by reading. **(b)** Goodput peaks at 40.3 TPS at concurrency 10. **(c)** What happens immediately after the peak: at concurrency 20 half the transactions fail and at 40 seventy per cent do. The knee is MVCC read-write conflict on the same keys, not queueing — so it is a property of how the chaincode writes, and it is the number to quote.

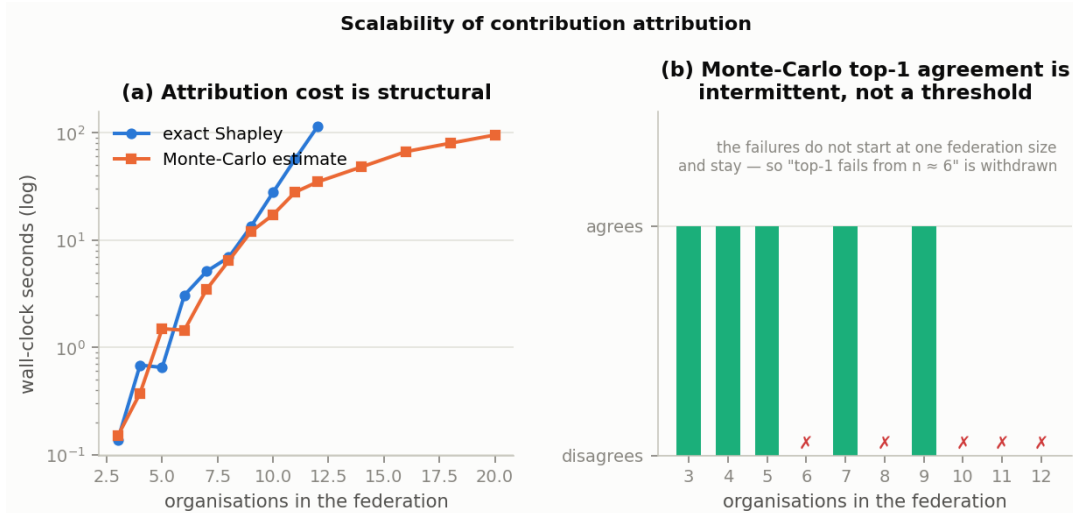


Figure 14: Contribution attribution. **(a)** Exact Shapley cost grows with the coalition count and the Monte-Carlo estimator is barely a speed-up in the range we can check — the cost is structural, which is confirmation rather than news. **(b)** Monte-Carlo top-1 agreement against the exact values: it fails at $n = 6$, agrees at 7 and 9, fails at 8, 10, 11 and 12. There is no federation size after which it stays wrong, which is why “MC top-1 fails from $n \approx 6$ ” is withdrawn in Section 11 rather than restated with a different threshold.

11. What may be quoted, and what may not

Several numbers in older drafts, and in the brief the supervisor has already read, are no longer safe to use. Some are simply wrong; others depend on a choice we made arbitrarily and could just as easily have made differently. **If you are writing anything at all — a slide, a paragraph, an**

email — read this list first. The left column is what not to say; the right column is what to say instead.

Do not say this	Say this instead — and here is why
“Krum costs 1.5–12.6 pp when banks hold separate customers”	The explanation was falsified by our own control (Section 7.6), and part of the range came from a contaminated score. Say: Krum catches the attacker 8/8 in all three conditions ; it can cost accuracy by an amount that tracks the dataset; we do not yet know why . If you need numbers, use the uncontaminated cells only: -4.73 to -1.48 pp (separate customers), -9.15 to $+0.54$ pp (mixed).
“a $60\times$ privacy improvement” — or $30\times$, $33\times$, or $\geq 100\times$	All four of these are the same data read with different arbitrary cut-offs. The number is not stable, so no version of it is safe. Say instead: the ordering (the output channel wins at 12 of 12 tested settings) and the ranking gap at a stated noise level . If a magnitude is truly unavoidable: “one to two orders of magnitude, and sensitive to definition”.
“the fast Shapley estimate fails from 6 banks onward”	There is no threshold. The failures are intermittent — $5/10$, $5/10$, $3/10$ — and on BankSim the first failure is at 3 banks, not 6. Say it is unreliable in a way that does not depend on size.
Any number out of <code>db_boa_results.json</code> or <code>baselines.json</code> , including MCC 0.677 as a tuned result	Invalid for three separate reasons: a search that could not rank its own candidates, a stand-in model tested on rows it had memorised, and the unexplained ULB reproduction failure. The first two are now fixed; the third is not. Regenerate before quoting anything, and state the third limitation explicitly.
MCC 0.313, 97.38% accuracy, MCC 0.966, 85 TPS, 180 ms	All withdrawn — either before submission or during the multi-run work. The detector headline is 0.753 ± 0.055 over five runs ; the blockchain figures are 2115.9 ms and 40.3 TPS .
A repaired “steps per epoch” value quoted as a precise optimum	Quote the batch size it implies instead. Even repaired, that setting produces only 23 distinct batch sizes across 201 possible values, and a third of the range collapses into just two of them — so a precise-looking value is not precise.
Any ULB result generated before 1 September	Nothing yet. The reproduction failure is genuinely unexplained, so those numbers cannot be defended. BankSim is unaffected — both of its splits reproduced exactly.

Two habits that apply to everything in that table

- 1. Always name the score you are quoting (Rule 11).** Twice in these six days, two perfectly reasonable scores ranked the same result in *opposite* directions — two different measures of the Shapley estimator disagree about which dataset is better, and the privacy ordering scores 11 out of 11 on one measure and 10 out of 11 on another. Saying which one you used is not pedantry. **It is the difference between reporting a result and choosing one.**
- 2. Say “measured as far as we looked”, not “measured”.** Several numbers in this report get bigger the harder anyone searches — ε^* is the clearest case. That is a property of how the number is defined, not of the system, and it belongs written next to the number rather than discovered later by a reader.

12. What is left, and the plan to 19 September

Three things remain, and **the order they are in is not negotiable** — each one would be wasted work if done before the one above it.

#	What has to be done	Why it sits in this position	Cost
1	Re-run what the surrogate repair forces us to re-run. Regenerate the two now-invalid result files using the fixed pool, then re-run the five-seed detector test with the repaired configuration added as an extra arm.	Our detector headline ($+0.047$, $p = 0.34$) currently compares the default against a configuration that a <i>broken</i> search picked — so it is not really a comparison of anything. The code is already written. Read the note on the ULB reproduction failure first: a regenerated ULB number is only safe to quote if that third problem is fixed or stated explicitly alongside it.	3–5 h
2	The third dataset (Fraud Detection Handbook). Go/no-go on 12 September.	It has to come <i>after</i> item 1. Run it before, and its search results are produced by a search we already know is broken — and we pay for the whole thing twice.	loader+ about 11 h
3	Writing: set out the privacy threat model properly, and narrow the two title words that currently claim too much (B8).	Both were flagged as must-fix before publication, and both are writing rather than code, so neither is blocked by anything above.	writing only

Two commitments we are making in advance, so they cannot be renegotiated later

1. The 16–19 September writing block is protected. If the third dataset runs over, **the dataset gets cut, not the writing.** A third column that is only half-run is unusable on its own *and* it invites exactly the lopsided-table criticism Rule 8 exists to prevent. There is no version of this where we arrive at the deadline with results and no paper.

2. The 12 September go/no-go is decided on the date, not on how much we have already spent. If the Handbook is not producing sensible runs by then, it is dropped — regardless of the hours already in it. Writing this down now is the only way to stop sunk cost making the decision on the day.

12.1 Open questions we are carrying honestly

- Q1. Why does the 30 August ULB result not reproduce?** Everything reachable has been eliminated. The remaining candidate — the numeric library version at the time — is unrecoverable. Recorded as unexplained; guarded against recurrence.
- Q2. Does that reproduction failure reach the detector track?** Only one sweep was re-verified. Every ULB number generated before 1 September sits on a code and environment state that no longer exists. If a detector number moved the way the contribution split did, that is a Chapter 5 problem, not a paper-extension one. *If nothing moved, that is a clean reproducibility statement worth having in writing.*
- Q3. Why does Krum cost utility on BankSim?** The effect is measured and real; the mechanism we proposed is falsified. No replacement is on record, and inventing one would be the failure this whole period was about.
- Q4. Should the deployed model adopt the repaired batch-size formula?** Zero CPU to decide, but changing it invalidates every trained model. Deliberately still open.
- Q5. What do Shapley values look like under an entity-disjoint split?** Still unmeasured — the current near-uniform values $[0.167, 0.165, 0.165]$ are the reason the incentive layer has little to attribute. Not on the locked plan; no window before the deadline.

13. Closing assessment

Six days ago the work had one genuine contribution standing on one dataset, an unexamined search sitting at its centre, and a number quoted as measured that had never been measured.

It now has the same contribution standing on **two datasets and three conditions**, a search that has been examined thoroughly and whose bad news came out *stronger* for the examination, and a number we have deliberately stopped quoting because we proved it was not stable enough to print.

Three headline claims died along the way. All three were killed by tests we ran on ourselves, and in each case we had written down in advance what failure would look like before we ran the test. That advance writing is the whole difference between publishing a finding and answering an objection — the fact is identical, but who found it first is not.

The honest summary, for the supervisor

The work is smaller than it looked on 30 August, and considerably harder to argue with.

The contribution itself is intact. In plain terms: **protecting privacy by adding noise to the shared model weights destroys the calculation that decides who gets paid — and moving that noise to the published contribution score instead fixes the problem without giving up any privacy.** That result now holds on two datasets across three different federation setups, and it rests on two statements that do not depend on any arbitrary cut-off, rather than on a ratio that moved every time we redefined it.

Everything that did not survive is written down as not having survived, in Section 11, in the language a reader would need to check it.

A. Glossary

Every term this report uses, in the order a reader meets them. Nothing here assumes prior knowledge of the field.

Term	What it means, in plain language
<i>The system and its parts</i>	
Federated learning	Training one shared model across several organisations <i>without</i> any of them handing over their data. Each trains locally; only the trained numbers are shared.
FL-ADTCN	The whole system in this thesis: federated ADTCN detectors + differential privacy + Krum + Shapley + DB-BOA + reinforcement-learning leader election, on a Hyperledger Fabric blockchain. Six parts, six promises.
ADTCN	Our fraud detector. It reads a window of 10 consecutive transactions and looks for patterns across them. Measured honestly as a component: 5th of 7 on ULB, 7th of 7 on BankSim’s customer-linked windows. It is not a title claim and does not have to win.
FedAvg (Federated Averaging)	The standard way to combine what the banks send back, and the baseline everything is compared against. Take all the banks’ weights and average them number by number; send the average back to everyone; repeat. Simple and effective — but an average has no defence against a participant who lies, because averaging includes everybody by construction. That weakness is why Krum exists. See the box in Section 3.
Krum	The alternative to averaging. Instead of blending all the banks together, it picks the <i>single most typical</i> submission and uses only that, so a sabotaged one is never included. This is a <i>security</i> mechanism, not a fairness one.
Shapley value	A way of splitting credit fairly among collaborators, borrowed from game theory: how much worse would the result be if this bank had not taken part? Used here to decide token rewards. This is a <i>fairness</i> mechanism, not a security one.
DB-BOA	Dynamic Butterfly–Billiards Optimisation Algorithm. The automated search, taken from the base paper, that picks the detector’s settings. The framework is named after it; Section 7.9 is the audit of it.
Surrogate (the “stand-in” model)	The small, fast model DB-BOA trains in order to <i>score</i> a candidate setting. It is not the model that gets deployed, and most of Section 7.9 turns on keeping the two apart.
<i>Privacy and rewards</i>	
Differential privacy (DP), ε	Deliberate random noise added before sharing, so nothing shared can be traced back to an individual customer. ε is the dial: smaller ε = more noise = more privacy but a worse model.
Weight channel vs output channel	<i>Where</i> that noise gets added — to the model weights a bank shares, or to the contribution score it publishes. Choosing the second over the first is the thesis’s actual contribution.
Rank inversion	The reward ranking coming out in the wrong order once noise is added — i.e. a bank gets paid as though it contributed more than it really did.
ε^*	The heaviest noise setting at which we still <i>caught</i> a rank inversion. Section 7.8 explains why this number is not stable and why we stopped quoting it.
<i>Data and experiment design</i>	
ULB	Our first dataset. 284,807 real anonymised card transactions, 0.17% fraud, no customer column . Section 7.3.
BankSim	Our second dataset. 594,643 simulated transactions, 1.211% fraud, and crucially 4,112 identifiable customers , so a window can be one person’s own history.
Stratified vs entity-disjoint split	Two ways of dealing transactions out to the three banks. <i>Stratified</i> : shuffle everything and deal equally, keeping the fraud ratio the same — so the same customer can land in two banks. <i>Entity-disjoint</i> : deal out whole <i>customers</i> , so nobody’s history is split across banks. The second is far more realistic.

Term	What it means, in plain language (<i>continued</i>)
Window (global vs entity-linked)	The 10 transactions the detector reads at once. <i>Global</i> : whatever 10 rows came next in the file, probably 10 different people. <i>Entity-linked</i> : 10 transactions by the <i>same</i> customer. The difference between these two is worth more than any architecture choice we tested.
Confound control	An extra run that deliberately changes only <i>one</i> factor, so an effect can be pinned on a cause. The one in Section 7.6 cost three hours and killed our best claim.
Pre-registration	Writing down what you expect <i>and</i> what being wrong would mean, before running — then recording the result underneath without editing the prediction.
Common random numbers	Scoring every candidate on the <i>same</i> random sample, so the comparison between them is not corrupted by luck of the draw. The most transferable finding of the period (Section 7.9).

B. Where everything lives

The running state of the work is `TASK.md` at the repository root — objectives, pre-registrations, scorecards, and a verified-numbers table. Start at its `RESUME HERE` section. **Numbers live only in `db.boa.framework/results/*.json`**; generated write-ups live in `final_report_data/`. **Never hand-edit a generated draft** — regenerate it, which now takes seconds.

Script <code>db.boa.framework/experiments/</code>	(all under <code>Produces</code>)
<code>banksim.temporal_grid.py</code>	The entity-linked window grid (7 architectures $\times$ 2 orderings).
<code>basepaper_comparison.py</code>	The base-paper classifier and optimiser tracks, both datasets.
<code>detector_multiseed.py</code>	The 5-seed detector headline and the determinism probe.
<code>byzantine_robustness_sweep.py</code>	Krum at $n \geq 2f + 3$, four attack types.
<code>economic_byzantine_sweep.py</code>	Collusion, lone attackers, free-riders, isolation.
<code>private_incentive_sweep.py</code>	The privacy $\leftrightarrow$ incentive channels; takes <code>--eps-grid</code> and <code>--repeats</code> .
<code>scalability_sweep.py</code>	Exact vs Monte-Carlo Shapley cost and fidelity.
<code>sweeps_cross_condition.py</code>	The only correct way to compare conditions. Prints dataset effect and partition effect as separate columns and marks contaminated cells. Reads JSON only; seconds.
<code>objective_noise_audit.py</code>	The surrogate diagnosis. Deliberately pins the <i>pre-repair</i> behaviour.
<code>obj13_shipped_path_check.py</code>	The check that found the memorised-rows leak. Zero training.
<code>obj13_surrogate_repair.py</code>	The three-mode side-by-side and its held-out yardstick.
<code>make_report_figures.py</code>	Every figure in Sections 7 and 10, plus the score-index table and CSV. Reads stored results only — no training, no sampling, no new numbers — and prints the provenance of each panel. Regenerate with <code>python -m experiments.make_report_figures</code> ; outputs land in <code>results/figures/</code> .

Practical notes that cost us time to learn — read before running anything

- **Use the PowerShell runners, not the shell scripts.** On this machine `python` is not on Git Bash's path at all, and the one that *is* on the path does not have `torch` installed. You will get a confusing error rather than an obvious one.
- **Set `PYTHONIOENCODING=utf-8` when redirecting output, *and* open files with an explicit encoding.** These are two separate traps that produce similar-looking failures, and fixing the first does not fix the second.
- **Launch anything multi-hour detached, with a pre-flight check first.** The pre-flight asserts the code is in the state you think it is before spending the CPU. It costs seconds, and it is the only reason the 16-hour run finished with no surprises in it.
- **Every detector number is meaningless without a pinned torch version and thread**

count. This is not a formality — Section 7.1 is the story of what happens when they differ.